

ORIGINAL ARTICLE

Application of spiking neural networks and traditional artificial neural networks for solar radiation forecasting in photovoltaic systems in Arab countries

Bahgat Ayasi^{1,7}  · Iago X. Vázquez²  · Mohammed Saleh³  · Angel M. Garcia-Vico^{4,5}  · Cristóbal J. Carmona^{4,5,6} 

Received: 22 September 2024 / Accepted: 3 February 2025 / Published online: 1 March 2025

© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2025

Abstract

Incorporating photovoltaic (PV) systems into power grids is increasingly critical, especially in sun-rich Arab countries. This study introduces a novel approach using spiking neural networks (SNNs) for forecasting solar radiation. We utilize the Lava framework to develop and compare different neural architectures in both univariate and multivariate scenarios. Our findings reveal that SNNs achieve performance comparable to traditional artificial neural networks (ANNs) in modeling and forecasting. Specifically, the error rates of SNNs, when using a convolutional neural network (CNN) architecture on multivariate time series data, are similar to those of their ANN counterparts. However, SNNs present a significant advantage in power efficiency, being approximately 9 times more efficient when estimated theoretically on both the Loihi neuromorphic chip and traditional GPUs using the direct training approach with the SLAYER algorithm. Furthermore, the efficiency improvement is about 7 to 8 times on both Loihi and GPUs using the conversion training approach with a bootstrap algorithm. This research contributes to optimizing PV operations in Arab regions by introducing SNNs as a power-efficient alternative to traditional ANNs for solar radiation forecasting. It aligns with global sustainability goals by offering a robust model for efficiently predicting solar energy outputs while minimizing the computational energy footprint. Although the datasets in this study are drawn from sources in the Arab region, the methodologies and findings are applicable to other areas due to their universal relevance. The practical implications of our findings support the expansion of sustainable energy infrastructures, underscoring the strategic importance of innovative forecasting models in enhancing the reliability and efficiency of renewable energy sources.

Keywords Spiking neural networks (SNNs) · Time series forecasting · Neuromorphic computing · Solar energy prediction · Power efficiency · Photovoltaic (PV) system

1 Introduction

Palestine possesses significant solar energy potential, with approximately 3,000 h of sunshine annually and an average solar radiation of 5.4 kWh/m²/day [1]. This makes photovoltaic (PV) systems a strategic energy solution for achieving energy independence and fostering socioeconomic growth. It also aligns with sustainable development goal 7 (SDG 7) to ensure affordable, reliable, sustainable, and modern energy for all [2]. Previous studies have demonstrated the economic viability and technical benefits of PV systems in Palestine, including reduced grid losses, improved voltage levels, and decreased reliance on imported energy [3–5].



Moreover, deploying PV systems contributes to economic stability by reducing operational costs and minimizing dependence on expensive imported energy sources. Accurate solar forecasting enables better investment optimization, ensuring that PV installations are strategically placed to maximize returns, stimulating local economic growth and creating job opportunities in manufacturing, installation, and maintenance sectors [3, 6, 7]. Additionally, enhanced energy access through reliable solar power supports social development by improving living standards and enabling sustainable infrastructure development in remote and underserved areas [8, 9].

Optimizing PV system efficiency through advanced forecasting methods, such as SNNs and neuromorphic computing, can significantly impact the region's energy economy. By reducing the computational energy consumption of AI models, more generated solar energy can be allocated for public use, thereby increasing the net present value (NPV) of PV investments, which ranges from \$187 to \$2,811 per kW in Palestine [7]. Furthermore, while AI offers substantial opportunities to advance science, it poses significant challenges due to its high energy consumption through data centers and storage facilities supporting AI operations. This considerable energy usage raises critical questions about maximizing AI benefits while minimizing environmental impacts. Our study addresses these concerns by maintaining the accuracy and efficiency of solar radiation forecasting using SNNs and emphasizing the development of energy-efficient AI models. This dual approach ensures that deploying AI in renewable energy systems contributes positively to sustainable development goals without imposing additional environmental burdens. Additionally, efficient computational methods support sociopolitical challenges by promoting energy autonomy and sustainable infrastructure development [5, 6, 8–12]. Enhanced solar radiation forecasting is crucial for optimizing PV system efficiency and supporting energy independence in Arab nations [13–15]. This study focuses on hourly forecasting, essential for real-time energy management and grid stability. Studies have demonstrated the effectiveness of models like ARIMA in improving solar radiation forecasts, thereby aiding in integrating solar energy into national grids [16]. Additionally, addressing challenges such as reduced power factor in grid-tied PV systems requires comprehensive strategies to enhance the reliability and efficiency of renewable energy sources [17].

Time series forecasting in energy production has evolved from traditional statistical methods like ARIMA [18, 19] to deep learning techniques such as LSTM and CNNs, which offer improved predictive accuracy and better recognition of temporal patterns [20–23]. However, these deep learning models, while accurate, are often computationally intensive and consume significant amounts of energy, contributing to the overall energy footprint of AI operations [11, 24, 25]. This presents a challenge in regions like the Arab countries, where efficient use of solar energy is critical for socioeconomic development.

Integrating SNNs into the renewable energy sector presents an opportunity to optimize energy management in the Arab region. SNNs are highly power-efficient, mimicking the brain's minimal energy consumption, which is crucial for PV systems where computational energy usage impacts overall efficiency [25, 26]. By reducing computational energy consumption, SNNs maximize the energy output available for communities and align with global sustainability goals. Moreover, SNNs excel in modeling complex, nonlinear systems, making them ideal for accurate solar power forecasting, thereby promoting energy independence and environmental sustainability [26–36].

In this study, we implement a novel approach to solar radiation forecasting by leveraging neuromorphic computing principles using the Lava framework¹ [37], an open-source software platform for brain-inspired computing using SNNs. We develop and compare different network architectures in univariate and multivariate time series models, effectively harnessing the efficiency and scalability of SNNs, representing a significant advancement in renewable energy forecasting methodologies.

This research explores the potential of various SNN architectures and training strategies in PV forecasting, comparing their performance to traditional ANNs. By assessing these models using diverse datasets and learning algorithms, we aim to understand the trade-off between predictive accuracy and computational efficiency [38]. Our methodology demonstrates the significant benefits of power-efficient forecasting algorithms like SNNs in

¹ https://lava-nc.org/lava/notebooks/in_depth/tutorial01_installing_lava.html.

enhancing the efficiency and sustainability of PV systems, optimizing energy production and reducing fossil fuel dependency, thereby supporting economic development and sustainability in the Arab region.

This paper is organized as follows: Sect. 2 reviews related work, Sect. 3 describes the methodology, Sect. 4 presents the experimental setup, Sect. 5 discusses the results, and Sect. 6 concludes with implications and directions for future research.

2 Related work

This section discusses the primary concepts relevant to this study, focusing on time series forecasting and SNNs application.

2.1 Time series forecasting

Time series forecasting has evolved significantly, with traditional statistical methods like ARIMA models being foundational in modeling time-dependent data [39]. ARIMA models have been widely used for their ability to capture various patterns in stationary time series. Extensions of traditional ARIMA models have been explored to enhance forecasting accuracy. For instance, [40] proposed a model estimation of ARMA using genetic algorithms to forecast natural gas consumption, demonstrating improved predictive performance over classical ARMA models. Alternative approaches, such as gray prediction models, have also been employed in energy demand forecasting. [41] proposed improvements to gray prediction models using optimization algorithms like genetic algorithms and particle swarm optimization, resulting in more accurate energy demand forecasts.

The advent of machine learning and deep learning techniques has significantly improved forecasting accuracy, especially in handling nonlinear and complex patterns. Recurrent neural networks (RNNs) with long short-term memory (LSTM) units, introduced by [42], effectively address the vanishing gradient problem, enabling the capture of long-term dependencies in data. These models have surpassed traditional methods like ARIMA in various contexts due to their enhanced predictive capabilities [42–44].

Advanced deep learning models have been applied to environmental forecasting tasks. [45] addressed multi-hour and multisite air quality index forecasting using deep learning networks like CNNs and LSTMs combined with spatiotemporal clustering, enhancing predictive accuracy. However, these sophisticated models often require substantial computational resources, which may not align with energy-saving goals.

In the context of PV power production forecasting, hybrid methods combining deep learning models have shown promising results. [38] proposed a hybrid method combining deep time series clustering with feature attention-based deep forecasting, significantly reducing prediction errors. Similarly, [46] emphasized the effectiveness of hybrid approaches in accurately predicting PV output, although these models often demand advanced computational resources.

Further studies, such as [47–51], and [52], have explored various machine learning and deep learning methods for renewable energy forecasting, highlighting the trade-off between enhanced predictive capabilities and increased computational demands.

These studies illustrate the evolving landscape of renewable energy forecasting, showcasing the growing emphasis on neural networks and deep learning techniques. However, the considerable computational requirements of these advanced models can contradict the low-energy ethos of photovoltaic systems. This highlights a key area where SNNs can provide a significant advantage by offering comparable predictive power with lower energy and hardware demands.

Subsequently, our ANNs and SNNs displayed MAE, RMSE, and nRMSE values close to those reported in this literature, aligning with the challenges and innovations in solar radiation forecasting highlighted by [38, 53–56], and [57]. These studies collectively underscore the goal of enhancing accuracy and computational efficiency in renewable energy systems, establishing a benchmark that our research aspires to meet and contribute to.

2.2 Spiking neural networks (SNNs)

SNNs, recognized as the third generation of neural network models, are distinguished for their ability to emulate the temporal dynamics of biological neural networks [27]. Unlike earlier neural network models, SNNs utilize discrete events or “spikes” for communication between neurons, which occur when a neuron’s membrane potential exceeds a specific threshold.

The informational content conveyed by neurons is encoded in the number and timing of these spikes within the spike train rather than the form of each spike. This spike train consists of electrical pulses occurring at regular or irregular intervals, playing a crucial role in neural communication [58].

They are defined as postsynaptic potentials (PSPs), time-dependent functions reflecting changes in the postsynaptic neuron’s membrane potential. The response of a postsynaptic neuron to incoming spikes is modulated by these PSPs, which can be excitatory or inhibitory, depending on the type of synapse [59]. The PSPs are crucial for the transmission of signals across synapses and are mathematically modeled as in Eq. (1):

$$PSP(t) = \frac{t}{\tau} e^{(1-\frac{t}{\tau})} \quad (1)$$

where τ represents the time constant that characterizes the decay of the potential [60]. Alternatively, the effect of an input spike on the membrane potential of a postsynaptic neuron can also be expressed as in Eq. (2):

$$V_i(t) - V_{\text{rest}} = \delta i_j(t) \quad (2)$$

where $V_i(t)$ represents the membrane potential at time t , V_{rest} is the resting membrane potential, and $\delta i_j(t)$ represents the postsynaptic potential induced by a spike from a presynaptic neuron j [59].

2.2.1 Main neuron models in SNNs

- **Leaky Integrate-and-Fire (LIF) Model:** One of the simplest neuron models [59] described by Eq. (3):

$$\tau_m \frac{dV(t)}{dt} = -[V(t) - V_{\text{rest}}] + R \cdot I(t) \quad (3)$$

where $V(t)$ is the membrane potential, τ_m the membrane time constant, V_{rest} the resting potential, R the membrane resistance, and $I(t)$ the input current. When $V(t)$ reaches a given threshold, the neuron fires and resets $V(t)$ to V_{rest} .

- **Izhikevich Neuron Model:** Provides a computationally efficient framework for simulating neuronal firing patterns [61], governed by Eqs. (4) and (5):

$$\frac{dV}{dt} = 0.04V^2 + 5V + 140 - U + I \quad (4)$$

$$\frac{dU}{dt} = a(bV - U) \quad (5)$$

where V is the membrane potential, U the recovery variable, I the input current, and a , b , c , d are adjustable parameters. When $V \geq 30$ mV, the neuron spikes and resets V as in Eq. (6):

$$\text{if } V \geq 30 \text{ mV, then } \left\{ \begin{array}{l} V \leftarrow c, \\ U \leftarrow U + d \end{array} \right\} \quad (6)$$

This model efficiently simulates diverse neuronal behaviors.

- **Sigma-Delta ($\Sigma\Delta$) LIF Neuron Model:** Based on the Lava framework [37, 62], featuring a sigma decoder and delta encoder. The delta encoder activates based on ReLU outputs as in Eq. (7):

$$\Delta_t = \begin{cases} x_t - \hat{x}_{t-1} & \text{if } |x_t - \hat{x}_{t-1}| > \theta, \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

where x_t is the input at time t , $\hat{x}_{t-1}$ the previous estimate, and θ the threshold. The sigma unit reconstructs the input according to Eq. (8):

$$\hat{x}_t = \hat{x}_{t-1} + \Delta_t \quad (8)$$

This model reduces computational costs without increasing latency and supports various layer types.

- **Current-Based (CUBA) LIF Neuron Model:** Simplifies neuron dynamics by integrating synaptic currents directly into membrane voltage, represented by the differential Eq. (9) [37, 63]:

$$\tau_m \frac{dV(t)}{dt} = -V(t) + R \cdot I(t) \quad (9)$$

This model focuses on the direct relationship between current input and voltage response.

2.2.2 Learning paradigms in SNNs

Training SNNs is challenging due to the non-differentiable spike function. Our study utilizes two advanced methods:

- **Direct Training with SLAYER 2.0:** SLAYER (Spike Layer Error Reassignment in Time) [64] is a backpropagation mechanism for SNNs, using surrogate gradients to approximate spike derivatives. SLAYER 2.0 [37, 65] supports diverse neuron models like Sigma-Delta LIF and CUBA-LIF, enhancing flexibility and training efficacy.
- **ANN-to-SNN Conversion Using the Bootstrap Method:** Developed by [37], this method refines ANN-to-SNN conversion by iteratively tuning the ANN to mimic SNN outputs better, achieving efficient, low-latency SNNs with minimal performance loss.

These methodologies address training challenges and enhance computational efficiency and accuracy [64]. Other approaches include unsupervised learning through spike-timing dependent plasticity (STDP) and hybrid methods integrating neuromorphic and classical computing elements [66, 67].

SNNs are energy-efficient due to their spike-based communication, activating processing components only when necessary [27, 58, 59]. They excel in tasks involving temporal patterns, making them well-suited for dynamic environments where precision is crucial.

Recognized for their adaptability, SNNs are applied in signal processing and pattern recognition. They have been successful in financial forecasting [35, 68], image recognition [69–73], and renewable energy forecasting [32, 74, 75].

The study in [76] demonstrates SNNs' effectiveness in univariate time series forecasting using SLAYER, achieving low power consumption and fast processing suitable for edge devices. Their findings highlight SNNs' potential to outperform baseline models while maintaining low computational demands.

Recent work by [77] presents a nonlinear spiking mechanism-based model for short-term PV power forecasting using a spiking neural P system. However, it employs a single network architecture with rate coding and does not compute power consumption or model efficiency.

The rise in PV system installations increases the demand for reliable power generation forecasts. SNNs can analyze complex spatiotemporal data, improving PV forecasting accuracy and showcasing their broad applicability across various domains.

3 Materials and methods

Our research primarily aims to evaluate the efficacy of SNNs in predicting time series data for PV systems. We utilized two hourly datasets from the National Solar Radiation Database [78]. One dataset originates from Jenin City, Palestine (latitude = 32.45, longitude = 35.3), and the other from Amman, Jordan (latitude = 31.97, longitude = 35.94), representing the Arab region.

3.1 Neural network description

Our study aims to enhance the energy efficiency and accuracy of time series forecasting using neural networks. We explore the capabilities of SNNs alongside traditional ANNs, focusing on leveraging SNNs' distinctive neuron dynamics and their potential advantages for energy-efficient processing in time series data analysis.

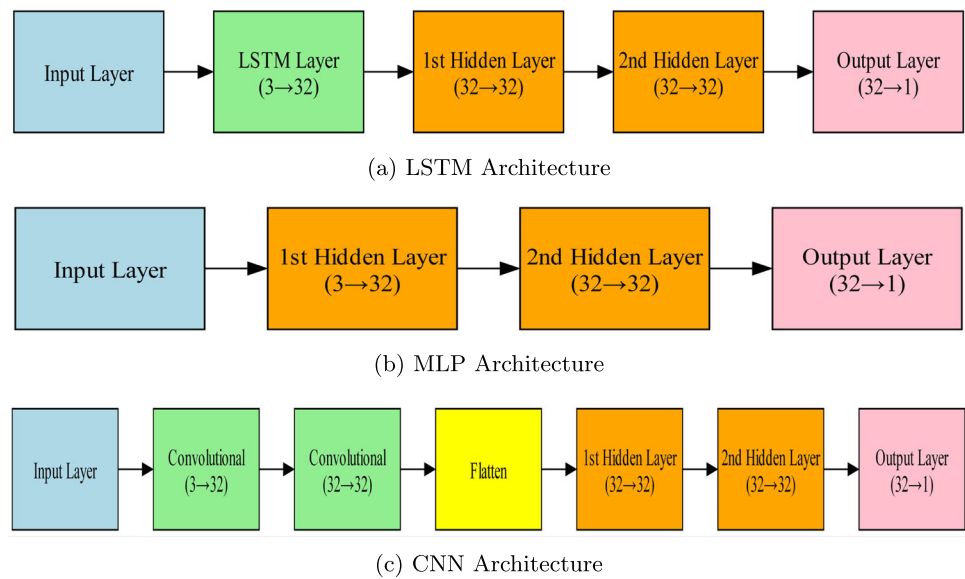
Specifically, our investigation evaluates the efficacy of various neural network models for forecasting univariate and multivariate time series data, including LSTMs, MLPs, and CNNs. We employ Sigma-Delta encoding [62] in SNNs, facilitated by direct training with the SLAYER surrogate gradient method, to handle the non-differentiable nature of spikes. This method allows us to maintain the temporal integrity of data, which is crucial for accurate time series forecasting. Additionally, we use rate coding in ANNs-to-SNNs training using the Bootstrap method [37]. This innovative conversion process minimizes latency and maintains high efficiency during inference, ensuring that ANNs can leverage SNN architectures without significant performance loss.

These approaches aim to prove the feasibility of SNNs in practical applications and demonstrate their superiority in energy efficiency compared to traditional ANNs, particularly in the context of time series analysis. We initiate our experiments with a basic configuration, starting with simpler network architectures. Initially, a single-layer network with the minimum viable number of neurons is deployed to determine baseline performance. Subsequently, the network complexity is systematically increased by adding more layers and neurons to each model-up to two hidden layers with 32 neurons each. This empirical approach ensures that each network configuration is optimized for stability and performance before adding further complexity.

To complement our study with an architecture capable of capturing long-term dependencies in time series data, we incorporated an LSTM model. The LSTM network, visualized in Fig. 1a and detailed in Table 7, was configured with a single LSTM layer consisting of 32 units to capture patterns from past and future data points effectively. This structure enhances the predictive accuracy of the network. Following the LSTM layer, our model includes two hidden (dense) layers with 32 neurons to refine the predictions. This LSTM configuration was empirically determined to balance training efficiency and computational demand, ensuring robust generalization across different datasets.

MLP and CNN models were chosen for their proficiency in capturing temporal and spatial patterns in data. The MLP model, visualized in Fig. 1b and detailed in Table 7, optimized for our study, consists of two hidden (dense) layers. Each hidden layer is equipped with 32 neurons, tailored to our data's characteristics, which include three time-lagged features derived from time series windowing. The CNN model, depicted in Fig. 1c and detailed in Table 7, is structured to enhance feature extraction, with two convolutional layers hosting 32 filters, a flattening operation, and two hidden (dense) layers. This configuration aids in the precise capture of localized, shift-invariant features crucial for time series analysis.

Fig. 1 Overview of network architectures for time series forecasting. **a** LSTM: one 32-unit layer plus two dense layers; **b** MLP: two dense layers with 32 neurons each; **c** CNN: two convolutional layers (32 filters) followed by two dense layers



Data encoding into spikes is critical for effectively integrating SNNs. For SNNs, we employ two methodologies: (1) **SLAYER Direct Training** with Sigma-Delta modulation and (2) **ANN-to-SNN Bootstrap Conversion**. Sigma-Delta modulation converts real-valued inputs into spike patterns, which are then processed using the SLAYER framework. This approach enables gradient-based learning despite the non-differentiability of spike events, preserving the temporal dynamics inherent in the input data and enhancing computational energy efficiency.

Conversely, the ANNs are trained using a traditional rate coding scheme. After training, these ANNs are adapted through a bootstrap method to facilitate their conversion into SNNs. This strategy ensures that both ANNs and SNNs operate under comparable conditions, enabling a fair comparison of their accuracy and energy consumption performance.

These methodological choices align with our goal of improving time series forecasts' energy efficiency and accuracy. The configurations of each network architecture used in our study are summarized in Table 1. The optimized architectures and training methods are discussed in detail in the results section, where we compare the performance outcomes of the different network configurations.

3.2 Data encoding in SNN for solar radiation forecasting

The accuracy and efficiency of SNNs depend heavily on the encoding mechanism used [79], especially in the context of solar radiation forecasting. Fluctuations, noise, and a significant presence of zero values characterize solar radiation datasets. Therefore, selecting an appropriate encoding strategy for effective forecasting is crucial. This study examines four encoding schemes implemented through Sigma-Delta LIF and CUBA-LIF neurons: rate coding, time-to-first-spike (TTFS), population encoding, and Sigma-Delta encoding. We will identify our

Table 1 Network architectures

Network type	Architecture	Neurons/filters
LSTM	Three layers: one LSTM layer, two dense layers	32 units in LSTM layer, 32 neurons (dense layers)
MLP	Two dense layers	32 neurons (dense layers)
CNN	Four layers: 2 convolutional, two dense layers	32 filters (convolutional layers), 32 neurons (dense layers)

dataset's most suitable encoding scheme in the experimental study detailed in Sect. 4. This scheme will be chosen based on its ability to demonstrate superior performance and achieve optimal model convergence. We will conduct this assessment for direct training and ANN-to-SNN conversion, considering the different neuron types used in our experiments.

3.2.1 Key encoding schemes employed

- **Sigma-Delta ($\Sigma\Delta$) Encoding in SNNs:** Integrates the difference between consecutive inputs ($\Delta x_t = x_t - x_{t-1}$) and triggers a spike when the accumulated sum of changes (Σ_t) and the current change (Δx_t) exceed a threshold, effectively encoding the input into spikes. This adaptive mechanism handles time-varying signals efficiently, significantly reducing energy consumption. Its ability to maintain high signal quality while minimizing power use is well-documented in neuromorphic computing [80].
- **Rate Coding in SNNs:** Encodes information through the frequency of spikes within a time window, defined as $V = \frac{N_{\text{spike}}}{T}$, where V is the mean firing rate, N_{spike} the number of spikes, and T the time window. Rate coding offers robustness against adversarial attacks and energy-efficient data processing, making it suitable for large-scale SNN applications [81].
- **Temporal Coding:** Particularly time-to-first-spike (TTFS) coding uses the precise timing of the first spike after a stimulus, given by Eq. (10):

$$t_{\text{spike}} = t_{\text{onset}} + \Delta t, \quad (10)$$

TTFS is biologically plausible and power-efficient, ideal for rapid sensory processing. However, it faces challenges in decoding complexity and sensitivity to noise, affecting robustness in hardware implementations [59, 82].

- **Population Coding:** Encodes information using the collective activity of neuron populations, represented as Eq. (11):

$$R = \sum_{i=1}^n w_i \cdot r_i \quad (11)$$

This method effectively handles noisy signals by distributing information across multiple neurons, enhancing robustness. It improves the linear separability of patterns, aiding classifiers like SVMs in achieving high accuracy in tasks like speech classification [83].

4 Experimental study

Our experimental study investigates a spectrum of model architectures across univariate and multivariate time series datasets, focusing on traditional ANNs, advanced SNNs, and ARIMA models as illustrated in the experimental workflow flowchart (see Fig. 2). We specifically evaluate SNNs for predicting time series data in photovoltaic (PV) systems. We utilized two distinct hourly datasets from the National Solar Radiation Database [78]. The datasets originate from Jenin City, Palestine (latitude = 32.45, longitude = 35.3), and Amman, Jordan (latitude = 31.97, longitude = 35.94), covering a period from 2017 to 2019. These datasets are crucial for training and validating our models for 1-hour ahead Global Horizontal Irradiance (GHI) forecasting, with 80% of the data allocated for training and the remaining 20% for testing. We include ARIMA models as benchmark traditional statistical models to provide a comparative baseline against our neural network approaches. The details of the meteorological parameters are summarized in Table 2.

Fig. 2 Flowchart of the Experimental Workflow. The process begins with dataset acquisition from the National Solar Radiation Database, followed by data preprocessing (normalization and time series windowing). Three model architectures (ANN, SNN, ARIMA) are developed and trained using specific encoding schemes (Sigma-Delta and Rate Coding). Models are evaluated on solar radiation forecasting accuracy using RMSE, MAE, and nRMSE, with energy efficiency metrics for SNNs to assess computational sustainability

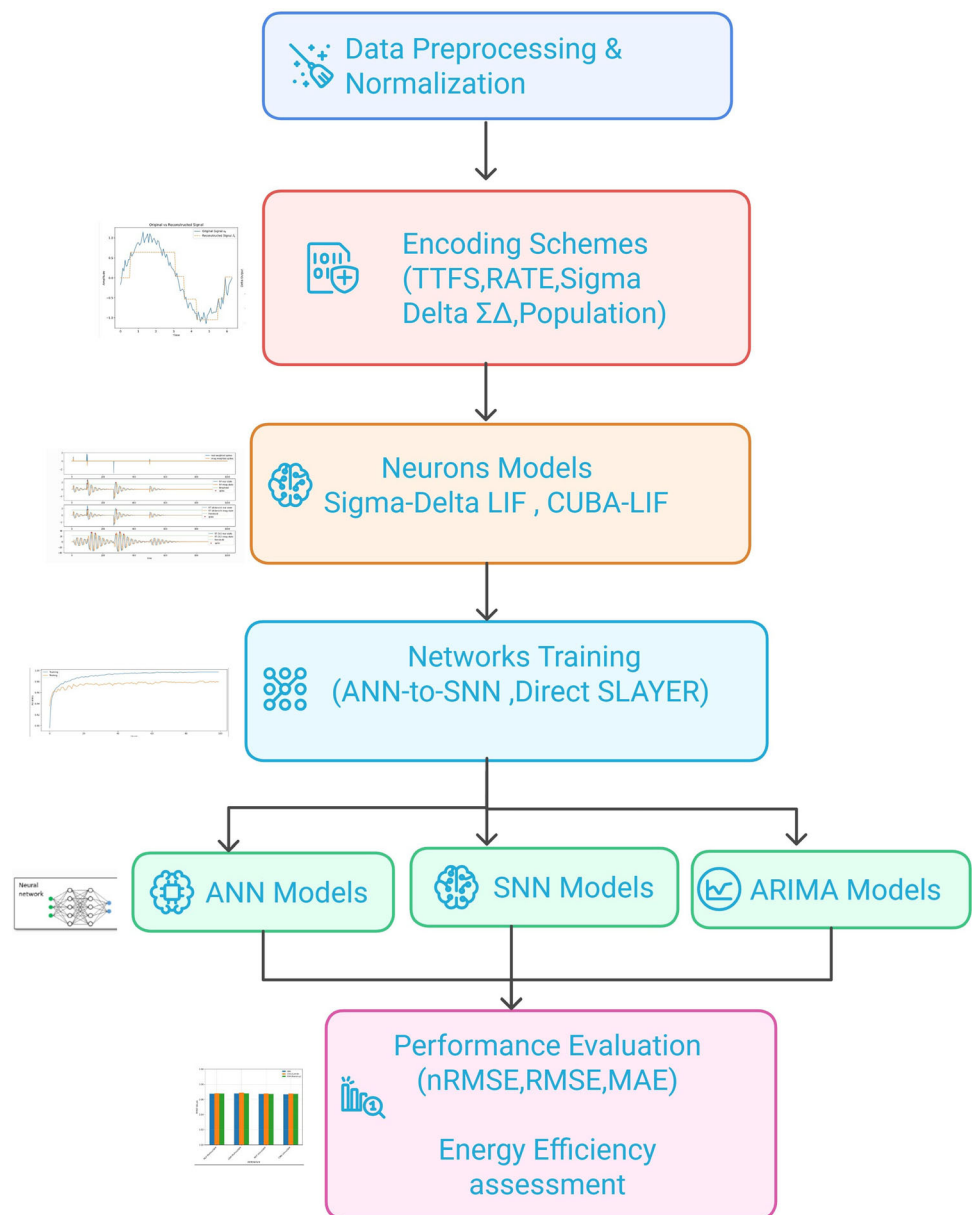


Table 2 Description of the dataset’s meteorological main features

Feature Name	Type	Description
Date	Date	Hourly time stamp
GHI	Float	Target label—Average Global Horizontal Irradiance (W/m ²)
Air Temperature	Float	Average air temperature in degrees Celsius
Dew Point	Float	The temperature at which air becomes fully saturated with moisture and dew forms, expressed in degrees Celsius
Relative Humidity	Float	Average relative humidity
Wind Direction	Float	Wind direction in degrees from true north
Wind Speed	Float	Maximum wind speed (meters per second)

Note: This table describes the main meteorological features of the dataset

In our experimental setup, we benchmark the performance of SNNs against traditional ANN architectures such as LSTM, MLP, CNNs, and ARIMA models. For the ARIMA models, we utilized the `auto_arima` function from the `pmdarima` Python library to automatically select optimal parameters, specifying a seasonal period of 24 to account for daily cyclical patterns. This approach balances model complexity and goodness of fit by minimizing the Akaike Information Criterion (AIC). Additionally, to enhance forecasting accuracy we incorporated exogenous variables—air temperature, dew point, relative humidity, wind direction, and wind speed—into the ARIMA models, resulting in ARIMAX models that provide valuable context for the forecasting task.

Our models incorporate two neuron dynamics models, Sigma-Delta LIF and CUBA-LIF, to evaluate their efficacy in univariate and multivariate time series scenarios. The ANN models are trained using traditional methodologies, while the SNNs are trained using two distinct approaches: direct training with SLAYER 2.0 and an improved Bootstrap method for ANN-to-SNN conversion. The Bootstrap method iteratively refines training to closely mimic the rate-based behavior of ReLU activations in traditional ANNs [37]. This structured approach ensures that both ANNs and SNNs operate under comparable conditions, facilitating a fair comparison of their predictive performance and computational efficiency.

Furthermore, ARIMA models were configured and evaluated in univariate and multivariate settings to serve as a robust benchmark against the neural network models. Model performance was assessed using consistent metrics across all models, including normalized root mean square error (nRMSE), root mean square error (RMSE), and mean absolute error (MAE), enabling a comprehensive comparison of their predictive capabilities.

4.1 Experimental framework

The datasets encompass meteorological parameters essential for modeling GHI, as detailed in Table 2. A comprehensive descriptive analysis was conducted on the meteorological datasets from Jenin and Amman, each comprising 26,280 observations. The findings are presented in Table 3 for Jenin and Table 4 for Amman. The Jenin dataset exhibits a mean air temperature of 21.23°C with a standard deviation of 7.60°C, indicating moderate temperature variability. The dew point averages at 12.86°C, while relative humidity maintains a mean of 63.86%, suggesting relatively consistent moisture levels. Wind direction in Jenin centers around 220°, with wind speeds averaging 1.60 m/s. Notably, solar radiation (GHI) in Jenin has a mean value of 221.70 W/m², reflecting substantial solar energy availability. In contrast, the Amman dataset records a slightly lower mean air temperature of 18.02°C and a higher standard deviation of 8.83°C, indicating more significant temperature fluctuations—dew point and relative humidity in Amman average 8.01°C and 59.97%, respectively. The wind direction in Amman is predominantly aligned at 234°, with higher wind speeds averaging 2.63 m/s. Solar radiation (GHI) in Amman shows a higher mean of 250.75 W/m² compared to Jenin, underscoring enhanced solar potential. These statistics

Table 3 Descriptive statistics of meteorological parameters in the Jenin dataset

Statistic	Air Temperature (°C)	Dew Point (°C)	Relative Humidity (%)	Wind Direction (°)	Wind Speed (m/s)	Solar Radiation (GHI) (W/m ²)
Count	26,280	26,280	26,280	26,280	26,280	26,280
Mean	21.23	12.86	63.86	220.06	1.60	221.70
Std	7.60	5.52	22.29	102.56	0.74	312.21
Min	1.50	−6.90	11.05	0.00	0.10	0.00
25%	15.00	8.90	44.55	134.00	1.00	0.00
50%	21.30	12.60	66.95	253.00	1.50	0.00
75%	26.30	17.30	83.85	302.00	2.10	418.00
Max	41.20	24.40	100.00	360.00	5.40	1,052.00

Table 4 Descriptive statistics of meteorological parameters in the Amman dataset

Statistic	Air Temperature (°C)	Dew Point (°C)	Relative Humidity (%)	Wind Direction (°)	Wind Speed (m/s)	Solar Radiation (GHI) (W/m ²)
Count	26,280	26,280	26,280	26,280	26,280	26,280
Mean	18.02	8.01	59.97	234.39	2.63	250.75
Std	8.83	5.85	27.54	86.59	1.49	333.92
Min	−3.30	−11.90	7.43	0.00	0.20	0.00
25%	10.90	4.30	35.66	155.00	1.40	0.00
50%	17.90	7.80	59.07	270.00	2.30	0.00
75%	24.00	12.10	85.45	300.00	3.60	513.00
Max	40.90	21.20	100.00	360.00	9.20	1,086.00

provide essential insights into the climatic conditions of both regions, informing subsequent solar radiation forecasting and energy optimization efforts.

With this foundation set, we move on to the specifics of the two analyses:

- **Univariate Time Series Analysis:** Focusing solely on the solar radiation metric, our univariate analysis was designed to predict based on past data points. The data at time “ t ” was influenced by preceding values at $t - 3$, $t - 2$, and $t - 1$. This method enabled predictions for subsequent hours using data from the last three time steps.
- **Multivariate Time Series Analysis:** We built the multivariate analysis upon the univariate approach and integrated five features (meteorological parameters) correlated to the target to predict solar radiation values. As with the univariate approach, we utilized past data from the three previous hours for forecasting.

Here, we outline the standard preprocessing steps employed for univariate and multivariate analysis.

- **Normalization:** Normalization was performed on all datasets, adjusting them to the 0-1 range using the min-max-scaler method available in the Python scikit-learn library.
- **Data Transformation for SNNs Input:** In preparing our SNNs dataset, we initiated specific data transformations to adapt the traditional time series data for supervised learning with SNNs.
- **Time Series Windowing:** We started by partitioning our time series data into sequences to serve as input features for our models. Specifically, we configured a sequence size (lag) of 3; the decision to use three lags is based on empirical evidence and is further elaborated in Sect. 5.4, implying that we used the current time step and the previous two-time steps as input features for every instance, aiming to predict the subsequent time step.

The efficacy of these models was rigorously quantified using three principal metrics: RMSE, MAE, and nRMSE. RMSE provides a measure of the average magnitude of prediction errors, calculated by the Eq. (12):

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (P_i - O_i)^2}, \quad (12)$$

where P_i denotes the predicted values, and O_i denotes the observed values. Complementarily, nRMSE, defined by the Eq. (13):

$$\text{nRMSE} = \frac{\text{RMSE}}{O_{\max} - O_{\min}}, \quad (13)$$

Normalizes RMSE relative to the observed data range, $O_{\max} - O_{\min}$, allowing for performance comparison across different scales of data. MAE evaluation metric defined by the Eq. (14):

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |P_i - O_i|, \quad (14)$$

MAE measures the average magnitude of the errors in a set of predictions without considering their direction. In addition to these accuracy metrics, energy efficiency is assessed through a comprehensive set of energy consumption metrics derived from the event-driven operational efficiency of SNNs. Energy consumption is evaluated based on the number of synaptic operations (*Synops*) and neuron updates during network inference, providing insights into the models' operational efficiency and sustainability. The analysis includes calculations of the energy per Synops (E_S) and energy per neuron update (E_N), integrating these metrics into our overall performance assessment framework. Furthermore, we employ three complementary methodologies to evaluate energy efficiency. **KerasSpiking-based energy estimation**, which provides high-level approximations of energy consumption using synaptic and neuronal operations, **A Synops-focused theoretical framework**, which quantifies energy usage based on hardware-agnostic Synops calculations and energy constants for MAC and spike-accumulation operations, and **Empirical GPU power measurements** using the *nvidia-smi* library [84], which captures real-time energy consumption during training and inference on a Tesla GPU T4. This multimetric evaluation enables a robust assessment of the model's predictive accuracy and applicability in energy-sensitive environments such as solar irradiance forecasting.

4.2 Experiment design and hyperparameters tuning

We optimize by exploring a straightforward SNN design with a single hidden layer. This baseline model helps us understand the minimal viable configuration for performance evaluation. Our architecture selection process is empirical, starting with a more straightforward setup and progressively increasing the complexity. We methodically add more layers, evaluating performance, learning stability, and convergence at each stage. Our experiments demonstrate that a two-hidden-layer configuration yields optimal results. Further increases in the number of layers did not significantly improve performance. Consequently, we adopted this two-layer setup across all neural network types-LSTM, MLP, and CNN, as detailed in Table 7. This structured approach ensures that our chosen architectures are well-suited for stability and performance, providing a robust basis for subsequent comparisons between ANN, SNN, and ARIMA models.

To provide specific details, the tuning process includes adjustments to learning rates, batch sizes, and neuron parameters (e.g., thresholds and decay rates). Each parameter is iteratively modified to ascertain its impact on the network's performance. The optimal parameters, as evidenced in Tables 5 and 6, result from this comprehensive evaluation. For ANNs, we use the Adam optimizer, while for SNNs, we utilize the RAdam optimizer [85], reflecting each model's unique requirements to achieve the most effective learning dynamics.

We used Python PyTorch computational frameworks for smooth model building and training in ANNs. Traditional backpropagation techniques facilitated practical model training. Table 7 provides a detailed overview of the architectures and configurations for ANNs, SNNs, and ARIMA models, including information such as the number of layers, neuron counts, activation functions, and other crucial settings for each type of network.

Table 5 Summary of Network Hyperparameters Tuning. This table presents a concise overview of the results from an extensive hyperparameters tuning study

Parameter	Range/options checked	Best parameters found for ANN and SNN
Neurons	10–100	32
Filter for CNN Layers	32–64	32
Activation	Relu, sigmoid, selu	Relu
Optimizer	Adam, RAdam, SGD, RMSprop	Adam in ANN and RAdam in SNN
Learning Rate	0.001–0.1	0.001
Batch Size	16–256	64
Epochs	20–120	60
Hidden Layers	1–4	2

This table summarizes the best hyperparameters found for both ANN and SNN models

Table 6 SNNs Neuron Hyperparameters Tuning

Neurons Type	Parameter	Range/options checked	Best parameters found
Sigma-Delta LIF	Neuron's threshold	0.01–0.9	0.05
Sigma-Delta LIF	tau_grad	0.01–0.9	0.05
Sigma-Delta LIF	scale_grad	1–3	1
Sigma-Delta LIF	requires_grad	True, False	True
Sigma-Delta LIF	shared_param	True, False	True
CUBA-LIF	Neurons threshold	0.01–1.5	0.5
CUBA-LIF	current_decay	0.03–1	1
CUBA-LIF	voltage_decay	0.03–0.25	0.03
CUBA-LIF	tau_grad	0.01–1	1
CUBA-LIF	scale_grad	1–3	1

Table 7 Network architectures and configurations

Network	Layers	Architecture	Neuron type	Encoding	Training
ARIMA	–	ARIMA(1,0,0)(1,1,1)[24]	–	None	Automated with auto_arima
ANN	LSTM	Three layers: one LSTM layer, two hidden layers	ReLU	None	Direct/Backpropagation
ANN	MLP	Two hidden (dense) layers	ReLU	None	Direct/Backpropagation
ANN	CNN	Four layers: 2 Convolutional, Two hidden (dense) layers	ReLU	None	Direct/Backpropagation
SNN	MLP	Two hidden (dense) layers	Sigma-Delta LIF	Sigma-Delta	Direct/SLAYER
SNN	CNN	Four layers: 2 Convolutional, Two hidden (dense) layers	Sigma-Delta LIF	Sigma-Delta	Direct/SLAYER
SNN	MLP	Two hidden (dense) layers	CUBA-LIF	Rate coding	ANN-to-SNN/Bootstrap
SNN	CNN	Four layers: 2 Convolutional, Two hidden (dense) layers	CUBA-LIF	Rate coding	ANN-to-SNN/Bootstrap

4.3 SNNs experiments, encoding scheme performance analysis, and models comparison framework

4.3.1 SNNs experiments

Our experiments implemented SNN architectures tailored to specific model requirements using the Intel LAVA framework's SLAYER 2.0 or the Bootstrap method for ANN-to-SNN conversion. We applied these training strategies across different network architectures—LSTM, MLP, and CNN—to assess their performance in univariate and multivariate time series datasets. All models, whether based on direct training or conversion techniques, were evaluated using a consistent data split of 80% for training and 20% for testing.

SLAYER 2.0 was specifically employed in networks where direct training of spiking models was feasible, enabling us to leverage its capacity to handle non-differentiable spike events effectively. Conversely, the Bootstrap method was utilized when a gradual transition from traditional ANN architectures to SNNs was necessary, ensuring minimal performance loss during the conversion process. Each training method's deployment was contingent on the nature of the task and the specific network type being used. Additionally, ARIMA models were trained and evaluated using similar data splits to maintain consistency in the comparison framework.

4.3.2 Encoding scheme performance analysis

To systematically evaluate the effectiveness of different encoding schemes for our SNN models, we conducted extensive experiments using rate coding, time-to-first-spike (TTFS), population encoding, and Sigma-Delta encoding. We assessed these schemes across multiple models (MLP and CNN) and training algorithms (SLAYER direct training with Sigma-Delta LIF neurons and ANN-to-SNN bootstrap conversion with CUBA-LIF neurons) on the Jenin and Amman datasets. The performance was measured using RMSE, MAE, and nRMSE. The detailed results are presented in Tables 8 and 9.

For models trained using the SLAYER direct training algorithm with Sigma-Delta LIF neurons, Sigma-Delta encoding with 100-time steps consistently achieved the lowest nRMSE and outperformed other encoding schemes in most evaluation metrics (RMSE and MAE). This indicates that Sigma-Delta encoding is highly effective in capturing the temporal dynamics of the datasets when used with SLAYER direct training. Conversely, for models trained using the ANN-to-SNN bootstrap conversion method with CUBA-LIF neurons, rate coding with 16-time steps yielded the best performance. It achieved the lowest nRMSE across most models and outperformed other encoding schemes in RMSE and MAE. This suggests that rate coding is more compatible with the bootstrap conversion method and CUBA-LIF neurons for our datasets.

Based on these empirical results, we selected Sigma-Delta encoding with SLAYER direct training for Sigma-Delta LIF neurons and rate coding with bootstrap conversion for CUBA-LIF neurons in our study. These selections are justified by their superior performance, as demonstrated in the evaluation metrics presented in Tables 8 and 9.

Table 8 Evaluation metrics for all SNN models using different encoding schemes over Sigma-Delta LIF neurons and the SLAYER direct training algorithm on Jenin and Amman datasets, bold values denote the best performance for each metric

Network	Time series	Encoding	RMSE Jenin	MAE Jenin	nRMSE Jenin	RMSE Amman	MAE Amman	nRMSE Amman
MLP	Uni	Sigma-delta	64	33	0.060	31.49	18	0.028
	Uni	Rate coding	88	50	0.084	75.00	45	0.069
	Uni	TTFS	67	33	0.064	43.00	24	0.040
	Uni	Population	62	31	0.061	31.00	18	0.029
CNN	Uni	Sigma-Delta	60	28	0.056	32.40	14.7	0.024
	Uni	Rate coding	78	60	0.082	74.00	45	0.068
	Uni	TTFS	65	37	0.062	43.00	24	0.040
	Uni	Population	60	29	0.056	29.00	16	0.027
MLP	Multi	Sigma-Delta	53	31	0.050	39.00	22	0.025
	Multi	Rate coding	74	50	0.071	63.00	45	0.058
	Multi	TTFS	57	30	0.055	39.00	23	0.036
	Multi	Population	55	27	0.052	29.00	15	0.026
CNN	Multi	Sigma-Delta	52	25	0.048	32.00	28	0.022
	Multi	Rate coding	74	50	0.070	63.00	46	0.057
	Multi	TTFS	57	34	0.055	40.00	24	0.037
	Multi	Population	54	26	0.052	32.00	19	0.027

Table 9 Evaluation metrics for all SNNs models using different encoding schemes over CUBA-LIF Neurons and ANN-to-SNN bootstrap conversion training algorithm on Jenin and Amman datasets, bold values denote the best performance for each metric

Network	Time series	Encoding	RMSE Jenin	MAE Jenin	nRMSE Jenin	RMSE Amman	MAE Amman	nRMSE Amman
MLP	Uni	Rate Coding	63.62	36	0.060	30.8	17	0.026
	Uni	Sigma-Delta	74	38	0.071	58	31	0.053
	Uni	TTFS	86	58	0.083	100	75	0.075
	Uni	Population	70	40	0.065	45	28	0.040
CNN	Uni	Rate Coding	65.32	30	0.057	31.75	17	0.025
	Uni	Sigma-Delta	78	41	0.074	61	33	0.055
	Uni	TTFS	79	52	0.080	88	68	0.072
	Uni	Population	76	48	0.059	44	24	0.038
MLP	Multi	Rate Coding	51.96	28	0.049	23.12	18	0.024
	Multi	Sigma-Delta	62	41	0.058	46	36	0.039
	Multi	TTFS	99	70	0.088	84	63	0.075
	Multi	Population	69	43	0.059	66	19	0.061
CNN	Multi	Rate Coding	50.90	29	0.048	22.60	15	0.022
	Multi	Sigma-Delta	58	40	0.057	44	31	0.036
	Multi	TTFS	89	68	0.080	76	59	0.071
	Multi	Population	65	41	0.057	52	17	0.060

4.3.3 Model comparison framework

This experimental setup allows for a direct comparison of ANNs, SNNs, and ARIMA models, crucial for evaluating their respective efficiencies in forecasting solar radiation. To ensure a fair comparison, similar architectures in terms of layer count and neuron density and, as far as practical, identical hyperparameters were employed across all models. This alignment is critical to ensuring that any observed performance differences are genuinely attributable to the model type (ANN vs. SNN vs. ARIMA) rather than external configuration variables. All of the coding and model training was conducted on Google Colab, utilizing the computational power of an NVIDIA Tesla GPU T4. Due to the high computational demands of training ANNs, SNNs, and ARIMA models, this setup ensured that our experimental results were reproducible and efficiently obtained. Additionally, the code used to run the experiments is available.²

5 Results and discussions

5.1 Models performance analysis

5.1.1 Detailed trainable parameters analysis:

Our analysis delves into the number of trainable parameters across different models, as detailed in Table 10. The number of trainable parameters in SNNs is typically lower than in their ANN counterparts despite using similar architectural frameworks. This discrepancy arises from the inherent differences in neuron modeling. SNNs employ spiking neurons, which do not require as many parameters per neuron as ANNs because the synaptic inputs and outputs are handled differently, often resulting in fewer weights but more state variables that are not trained but determined through the dynamics of the model itself. Additionally, SNNs can leverage temporal

² <https://github.com/bahgatayasi/Solar-Radiation-Forecasting-SNNs-ANNs-Source-Files.git>.

Table 10 Number of trainable parameters for each model configuration

Network	Type	Time series	Encoding	Trainable parameters
ANN	LSTM	Multi	None	8,801
ANN	LSTM	Uni	None	6,881
ANN	MLP	Multi	None	2,753
ANN	MLP	Uni	None	2,273
ANN	CNN	Multi	None	21,729
ANN	CNN	Uni	None	6,369
SNN	MLP	Multi	Sigma-Delta	1,640
SNN	MLP	Uni	Sigma-Delta	1,160
SNN	CNN	Multi	Sigma-Delta	3,692
SNN	CNN	Uni	Sigma-Delta	3,212
SNN	MLP	Multi	Rate	1,634
SNN	MLP	Uni	Rate	1,152
SNN	CNN	Multi	Rate	3,682
SNN	CNN	Uni	Rate	3,202

dynamics more effectively, reducing the need for large numbers of trainable parameters to achieve comparable levels of complexity and functionality. This effective reduction in trainable parameters can decrease computational overheads but may also impact the training dynamics, as seen in the variance in performance metrics across the datasets.

5.1.2 Models performance analysis and implications

The outcomes, detailed in Table 11, illustrate variations in RMSE and nRMSE that can be attributed to the differences in the characteristics of the Jenin and Amman datasets. For the Jenin dataset, RMSE values ranged from 48 for ANNs using a CNN architecture to 65.32 for SNNs utilizing a CNN with rate coding. Correspondingly, nRMSE values span from 0.047 to 0.060, reflecting subtle yet impactful differences in model precision. For the Amman dataset, RMSE values fluctuated between 17.37 and 39.09, while nRMSE values ranged from 0.016 to 0.028, indicating variations in model accuracy under different settings.

As detailed in Table 11, the ARIMA models generally exhibited higher RMSE and MAE values than the ANN and SNN models, indicating lower forecasting accuracy. These results demonstrate that neural networks are more effective at modeling the complex, nonlinear relationships inherent in solar radiation data than traditional statistical models like ARIMA.

Figures 3 and 4 visually compare the nRMSE across different network architectures and training methodologies, highlighting the nuanced performance metrics for both datasets. Specific observations from this analysis include:

- SNNs are adept at modeling the forecasting problem and understanding the intricacies and complexities of network operation. The spiking mechanism, inspired by biological neurons, allows SNNs to handle complex nonlinear changes in data effectively. This robust modeling capability makes SNNs suitable for tasks requiring high computational integrity and nonlinear dynamics like solar radiation forecasting.
- For univariate and multivariate time series, ANNs with CNN, LSTM, and MLP architectures delivered similar performance for the Jenin dataset. Notably, the SNN network in CNN multivariate series with Sigma-Delta LIF and Sigma-Delta coding using SLAYER direct surrogate training performs comparably to the ANNs (MLP, CNN, LSTM) on the Jenin dataset. Additionally, the SNN network with a bootstrap ANN-to-SNN indirect learning approach in CNN multivariate series with CUBA-LIF neurons and rate coding also performs similarly to the same ANNs on the Jenin dataset.

Table 11 Experimental results comparing three groups of models for solar radiation forecasting: ANNs, SNNs trained using the SLAYER method, and SNNs trained using the Bootstrap method. The best-performing model within each group is highlighted in bold, based on the minimum normalized RMSE (nRMSE) error for each dataset (Jenin and Amman)

Network	Layers	Time series	Neurons	Encoding	Training	RMSE Jenin	MAE Jenin	nRMSE Jenin	RMSE Amman	MAE Amman	nRMSE Amman
ARIMA	–	Uni	–		Automated with auto_arima	58.00	32.0	0.055	35.00	22.0	0.032
ARIMA	–	Multi	–	None	Automated with auto_arima	65.00	48.0	0.062	26.00	26.0	0.034
ANN	LSTM	Multi	ReLU	–	Backprop	50.00	27.8	0.047	18.00	12.7	0.017
ANN	LSTM	Uni	ReLU	–	Backprop	57.80	31.0	0.053	23.00	13.7	0.020
ANN	MLP	Multi	ReLU	–	Backprop	49.80	27.0	0.047	17.37	16.0	0.018
ANN	MLP	Uni	ReLU	–	Backprop	57.30	31.6	0.053	23.02	14.0	0.022
ANN	CNN	Multi	ReLU	–	Backprop	48.34	24.0	0.047	18.46	12.7	0.016
ANN	CNN	Uni	ReLU	–	Backprop	59.80	30.0	0.053	26.06	23.0	0.021
SNN	MLP	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	53.00	31.0	0.050	25.09	22.0	0.025
SNN	MLP	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	64.00	33.0	0.060	31.49	18.0	0.028
SNN	CNN	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	52.00	25.0	0.048	22.58	28.0	0.022
SNN	CNN	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	60.00	28.0	0.056	24.10	14.7	0.024
SNN	MLP	Multi	CUBA-LIF	Rate coding	Bootstrap	51.96	28.0	0.049	23.12	18.0	0.024
SNN	MLP	Uni	CUBA-LIF	Rate coding	Bootstrap	63.62	36.0	0.060	30.83	17.7	0.026
SNN	CNN	Multi	CUBA-LIF	Rate coding	Bootstrap	50.90	29.0	0.048	22.60	15.0	0.022
SNN	CNN	Uni	CUBA-LIF	Rate coding	Bootstrap	65.32	30.0	0.057	31.75	17.0	0.025

- For univariate series, SNNs with CNNs showcased a competitive edge over MLPs, particularly evident in the Amman dataset.
- SNNs with Sigma-Delta LIF neurons, when trained directly using SLAYER 2.0, exhibited robust training and convergence, especially noticeable in the multivariate series for both datasets.
- Conversion from ANNs to SNNs using rate coding and CUBA-LIF neurons proved effective, as highlighted in the results for both datasets.

Regarding the RMSE, it is crucial to contextualize its significance within the specific framework of solar radiation forecasting. This metric quantifies the average magnitude of the prediction errors, indicating the model's accuracy in estimating GHI. Understanding the RMSE values is essential as they directly reflect the precision with which the models can predict solar energy availability, a critical factor in optimizing and assessing the efficiency of photovoltaic systems.

- An RMSE of 17.37 for the Amman dataset signifies that the average error in solar radiation (GHI) predictions is approximately 17.37 units. Given that this dataset features values ranging widely across daylight hours, an

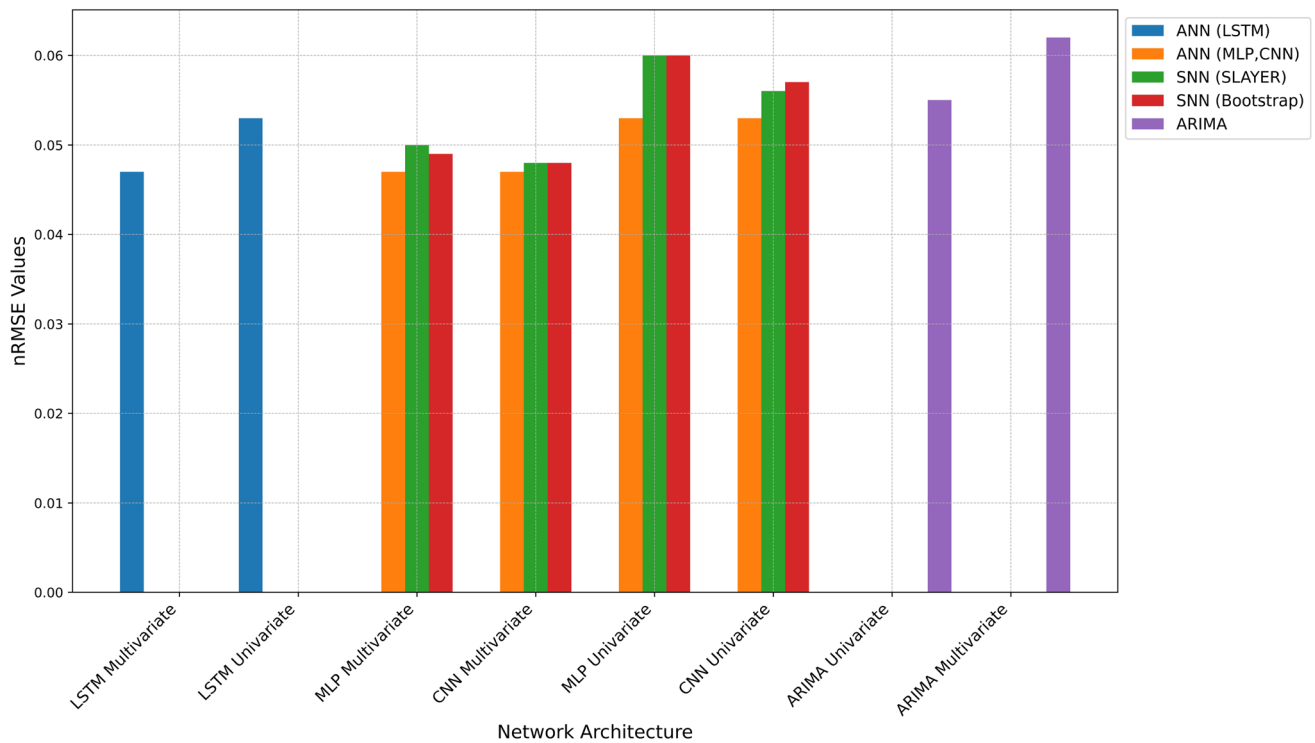


Fig. 3 nRMSE by Network Architecture and Training Methodology (Jenin Dataset)

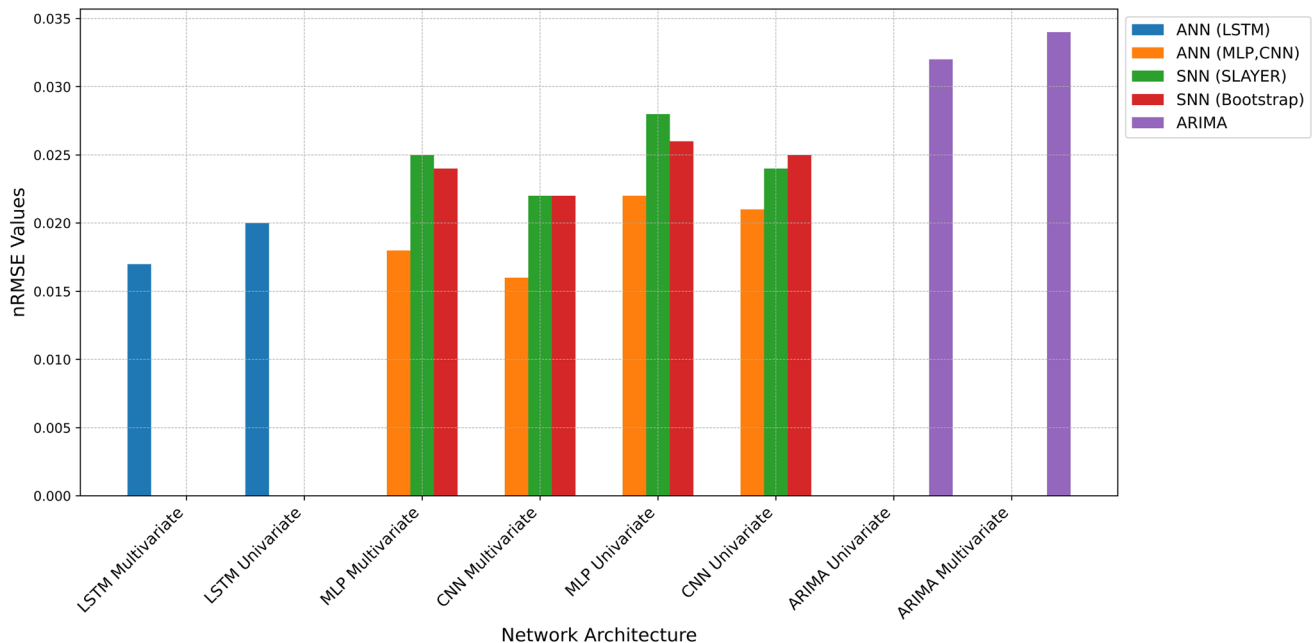


Fig. 4 nRMSE by Network Architecture and Training Methodology (Amman Dataset)

RMSE of this magnitude indicates a relatively high accuracy, considering the variability in solar radiation throughout the day.

- For the Jenin dataset, an RMSE of 48 represents the model's average error in predicting solar radiation. While higher than the Amman dataset, considering the environmental and atmospheric conditions that may vary significantly in Jenin, this value still reflects a reasonable prediction accuracy.

While our findings demonstrate that SNNs, with their intrinsic event-driven processing, have significant potential, it is clear that they do not uniformly match the performance of ANNs across all scenarios. The performance of SNNs notably depended on the choice of encoding schema and neuron model. This suggests that SNNs, while potentially rivaling ANNs, require meticulous parameter optimization and careful architectural decisions to achieve optimal performance. Although SNNs do not beat ANNs in accuracy metrics, they are much more efficient in parameters and energy consumed, as will be proven in the energy analysis in the next section. The comparative effectiveness of these models indicates that although advanced mechanisms can enhance performance, as noted in the related work section, the simpler models used in our research also produce robust predictions. This is particularly important for applications such as energy management in smart grids, where balancing computational demand with operational efficiency is critical. It shows that significant advancements in renewable energy forecasting can be achieved even with more straightforward technological implementations.

Moving forward, we will explore the interplay between performance, power consumption, and network sparsity and discuss energy efficiency further, which is crucial for the practical deployment of these networks.

5.2 Analysis of sparsity metrics

The comparison presented in Table 12 underscores the substantial advantages of using SNNs over ANNs in terms of computational sparsity. We calculated the sparsity of the SNNs empirically by analyzing the network models during inference on the testing datasets. Specifically, we measured the number of neuron activations (events) and Synops in the SNNs. We compared them to an iso-architecture ANN—an ANN with the same architecture but without spiking behavior. We ran the trained SNN models on the test data to quantify the sparsity and counted the actual events (spikes) and Synops during inference. These counts were averaged over the test samples to obtain the mean activity levels. By comparing these empirical counts to the total possible activations and Synops in the equivalent ANN, we derived the **Event Reduction Factor** and **Synops Reduction Factor**, quantifying the reductions in neuron activations and synaptic operations. In our experiments, networks using Sigma-Delta LIF neurons with Sigma-Delta encoding exhibited significantly higher sparsity levels than those using CUBA-LIF neurons with rate encoding. This is evident from the Event Reduction Factors and Synops Reduction Factors listed in Table 12, where the highest reductions were observed in the Sigma-Delta configurations. These results demonstrate that SNNs activate far fewer neurons and perform significantly fewer synaptic operations than their ANN counterparts. This increased sparsity suggests that SNNs can minimize computational overhead, leading to reduced energy consumption—a benefit we explore further in the next section.

5.3 Comprehensive analysis of energy efficiency in SNNs vs. ANNs

In this section, we examine the energy efficiency of SNNs compared to ANNs using three complementary methodologies. By integrating KerasSpiking-based estimates (Methodology #1), a Synops-focused theoretical framework (Methodology #2), and accurate GPU power measurements (Methodology #3), we provide a robust, multiperspective analysis of SNN energy consumption across diverse hardware and modeling paradigms.

5.3.1 Methodology #1 (KerasSpiking Energy Estimation).

In our first approach to evaluating energy efficiency, we followed the methodology of [86], which relies on the **KerasSpiking** library [87] for estimating the per-inference energy consumption of both *SNNs* and *ANNs*. KerasSpiking uses high-level abstractions of synaptic and neuronal operations and integrates empirical or literature-based constants to yield approximate energy estimates. These hardware constants stem from the documentation provided by KerasSpiking itself [87] and are based on published, though generalized, device specifications. The KerasSpiking framework seamlessly integrates with TensorFlow/Keras, offering a streamlined workflow that simplifies model training, inference, and energy estimation. By incorporating the “firing events”

Table 12 Comparison of event and Synops sparsity between SNNs and ANNs for Jenin and Amman datasets, with bold values highlighting the highest reductions and, hence, the most significant efficiency gains

Dataset	Network	Layers	Time series	Neurons	Encoding	Training	SNN event reduction factor	SNN Synops reduction factor
Jenin	SNN	MLP	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	6.68x	4.62x
	SNN	MLP	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	34.88x	29.93x
	SNN	CNN	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	10.34x	8.95x
	SNN	CNN	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	28.87x	28.96x
	SNN	MLP	Multi	CUBA-LIF	Rate coding	Bootstrap	3.41x	4.0x
	SNN	MLP	Uni	CUBA-LIF	Rate coding	Bootstrap	5.35x	5.61x
	SNN	CNN	Multi	CUBA-LIF	Rate coding	Bootstrap	6.2x	7x
	SNN	CNN	Uni	CUBA-LIF	Rate coding	Bootstrap	8.15x	9.6x
	SNN	MLP	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	6.70x	4.75x
Amman	SNN	MLP	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	36.19x	33.64x
	SNN	CNN	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	10.27x	9.09x
	SNN	CNN	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	32.61x	36.38x
	SNN	MLP	Multi	CUBA-LIF	Rate coding	Bootstrap	4.71x	5.59x
	SNN	MLP	Uni	CUBA-LIF	Rate coding	Bootstrap	6.15x	6.47x
	SNN	CNN	Multi	CUBA-LIF	Rate coding	Bootstrap	7.20x	8.5x
	SNN	CNN	Uni	CUBA-LIF	Rate coding	Bootstrap	12.6x	14.1x

concept in SNNs, the framework effectively accounts for potential energy savings through event-driven operations. Moreover, KerasSpiking is well-aligned with existing research, as evidenced by its use in various studies [86, 88, 89]. These studies have leveraged KerasSpiking to assess the energy efficiency of SNNs and to benchmark their performance against ANNs, particularly in smaller-scale research projects and proof-of-concept analyses. Although KerasSpiking provides a convenient high-level tool for energy estimation, it lacks rigorous, peer-reviewed validation for hardware-specific accuracy. Its documentation explicitly warns that these predictions should be treated as “*very rough estimates only*,” with real-world consumption influenced by device architecture, data transfer overhead, and other factors. Hence, we regard these numbers as a *conceptual baseline*, supplementing them with more detailed theoretical (Sect. 5.3.2) and empirical (Sect. 5.3.3) methods.

Energy Consumption Formula. Building on the notion of SNN *sparsity* (i.e., fewer active synaptic events than dense ANN computations), we adopt KerasSpiking’s conceptual framework to estimate the approximate energy cost of each inference. Following these prior studies [86–89], we define two core energy components:

- **Synops Energy (E_S):**

$$E_S = E_o \times S \quad (15)$$

where E_o is the energy per *synaptic operation* (Synops) and S is the total number of synaptic events during inference. In SNNs, each synaptic event corresponds to a spike reception; in ANNs, it parallels each multiply-and-accumulate (MAC) operation.

- **Neuron Updates Energy (E_N):**

$$E_N = E_u \times U \quad (16)$$

where E_u denotes the energy cost per neuron update, and U is the total number of membrane updates (SNNs) or neuron activations (ANNs).

- **Total Energy Consumption (E_{total}):**

$$E_{total} = E_S + E_N. \quad (17)$$

Table 14 presents typical values for E_o and E_u (in Nanojoules) across two hardware paradigms—*traditional GPUs* (e.g., NVIDIA GTX Titan Black) and the *Loihi* neuromorphic platform—as used in [86, 87]. These constants may be adapted as more hardware-specific data becomes available.

5.3.2 Results and discussion

Despite the approximate nature of the KerasSpiking model, our estimates **consistently indicate lower energy consumption for SNNs** relative to ANNs across various benchmarks. Table 13 shows that SNN configurations can provide up to a $\sim 9\times$ reduction in per-inference energy cost when run *conceptually* on neuromorphic hardware (e.g., Loihi), with a more modest—though still significant—improvement on GPUs. Closer inspection in Table 13 reveals that an ANN CNN Multimodel on the Jenin dataset consumes around 1,148 nJ on an NVIDIA GPU and 111 nJ on Loihi, whereas its SNN counterpart (using Sigma-Delta LIF neurons and SLAYER training) expends only 127.61 nJ on the GPU and 12.30 nJ on Loihi—representing a $9\times$ improvement. Similar savings emerge on the Amman dataset, underscoring how neuromorphic platforms (designed for event-driven processing) can amplify SNN energy advantages. **Notably, these $9\times$ and $7\text{--}8\times$ gains arise when comparing the SNN CNN Multimodel (trained with SLAYER or Bootstrap)—the best-performing configurations per the table—against the corresponding ANN CNN Multimodel.** SNNs can exploit *event-driven* computations, decreasing the number of active synaptic operations relative to dense ANN workloads. Conventional GPUs (based on von Neumann architectures) cannot fully leverage spiking-based sparsity. As a result, the energy benefits of SNNs are more pronounced on specialized neuromorphic hardware like Loihi. Improvements of $\sim 7\times$ to $\sim 9\times$ are particularly relevant for edge computing or photovoltaic-based systems, where power availability is limited. While these results from KerasSpiking reinforce the case for SNN energy efficiency, they are complemented by more precise theoretical and empirical methods: (i) **Methodology #2:** A *theoretical Synops-based* calculation (see Sect. 5.3.2), following [92], to approximate energy based on hardware-level multiply-accumulate (MAC) constants, and (ii) **Methodology #3:** *Empirical GPU measurements* (see Sect. 5.3.3) following [93], capturing real power usage on a Tesla GPU T4 via the *nvidia-smi* library [84]. In summary, **Methodology #1** offers a convenient, high-level evaluation of SNN energy usage using published constants and approximate modeling. The subsequent sections introduce more rigorous methods for hardware-specific validation, ensuring a thorough and transparent energy efficiency assessment.

Table 13 Estimated energy consumption on hardware platforms for inference of a single sample (in Nanojoules) between SNNs and ANNs, bold entries denote the lowest energy estimates, illustrating the efficiency gains of SNNs over ANNs

Dataset	Network	Layers	Time series	Neurons	Encoding	Training	Ettotal on GPU (nJ)	Ettotal on Loihi (nJ)
Jenin	ANN	LSTM	Multi	ReLU	-	Backprop	2,631	243
	ANN	LSTM	Uni	ReLU	-	Backprop	2,055	191
	ANN	MLP	Multi	ReLU	-	Backprop	514	50
	ANN	MLP	Uni	ReLU	-	Backprop	366	36
	ANN	CNN	Multi	ReLU	-	Backprop	1,148	111
	ANN	CNN	Uni	ReLU	-	Backprop	999	97
	SNN	MLP	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	109.76	10.59
	SNN	MLP	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	12.52	1.24
	SNN	CNN	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	127.61	12.30
	SNN	CNN	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	33.44	3.27
	SNN	MLP	Multi	CUBA-LIF	Rate coding	Bootstrap	129.74	13.04
	SNN	MLP	Uni	CUBA-LIF	Rate coding	Bootstrap	65.46	6.60
	SNN	CNN	Multi	CUBA-LIF	Rate coding	Bootstrap	163.23	16.03
	SNN	CNN	Uni	CUBA-LIF	Rate coding	Bootstrap	104.29	10.30
Amman	ANN	LSTM	Multi	ReLU	-	Backprop	2,631	243
	ANN	LSTM	Uni	ReLU	-	Backprop	2,055	191
	ANN	MLP	Multi	ReLU	-	Backprop	514	50
	ANN	MLP	Uni	ReLU	-	Backprop	366	36
	ANN	CNN	Multi	ReLU	-	Backprop	1,148	111
	ANN	CNN	Uni	ReLU	-	Backprop	999	97
	SNN	MLP	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	106.85	10.32
	SNN	MLP	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	10.84	1.08
	SNN	CNN	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	125.69	12.13
	SNN	CNN	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	27.60	2.71
	SNN	MLP	Multi	CUBA-LIF	Rate coding	Bootstrap	92.91	9.35
	SNN	MLP	Uni	CUBA-LIF	Rate coding	Bootstrap	56.73	5.72
	SNN	CNN	Multi	CUBA-LIF	Rate coding	Bootstrap	135.69	13.36
	SNN	CNN	Uni	CUBA-LIF	Rate coding	Bootstrap	70.93	6.97

5.3.3 Methodology #2: Synops-based theoretical energy estimation

We implemented a theoretical methodology based on the framework proposed in [92]. This approach quantifies synaptic operations (Synops) as the principal driver of energy usage in SNNs, in contrast to the multiply-and-accumulate (MAC) operations typically used in ANNs. We derive a hardware-agnostic but principled approximation of energy consumption by calculating Synops for each layer and multiplying by known energy constants for both MAC and spike-accumulation operations.

5.3.4 Synaptic operations calculation

Following [92] each SNN layer l requires according to the Eq. (18):

$$\text{Synops}(l) = f_r \times T \times \text{FLOPs}(l) \quad (18)$$

where f_r is the firing rate (spikes per neuron per time step), T is the total number of time steps for the layer's activity, and $\text{FLOPs}(l)$ represents the floating-point operations characteristic of that layer. Our experiments measured firing rates by counting spikes within each layer during inference.

5.3.5 Theoretical energy computation

After computing Synops for each layer, we applied the energy model from [92], which factors in: - E_{MAC} for the first layer's MAC operations, and - E_{AC} for subsequent spike-accumulation (AC) operations.

Formally, for an SNN with N convolutional (Conv) layers and M fully connected (FC) layers, the total SNN energy consumption is shown in Eq. (19):

$$E_{\text{SNN}} = E_{\text{MAC}} \times \text{FLOP}_{\text{SNNConv}}^1 + E_{\text{AC}} \times \sum_{n=2}^N \text{Synops}_n^{\text{SNNConv}} + \sum_{m=1}^M \text{Synops}_m^{\text{SNNFC}}, \quad (19)$$

where $\text{FLOP}_{\text{SNNConv}}^1$ denotes the FLOPs in the first convolutional layer, and $\text{Synops}_n^{\text{SNNConv}}$, $\text{Synops}_m^{\text{SNNFC}}$ represent the Synops in subsequent layers.

For SNNs composed solely of M FC layers (no convolutional layers), the total energy becomes as in Eq. (20):

$$E_{\text{SNN-FC}} = E_{\text{MAC}} \times \text{FLOP}_{\text{SNNFC}}^1 + E_{\text{AC}} \times \sum_{m=2}^M \text{Synops}_m^{\text{SNNFC}}, \quad (20)$$

where $\text{FLOP}_{\text{SNNFC}}^1$ denotes the MAC-based operations in the first fully connected layer, and the remaining layers rely on spike-based AC operations.

Meanwhile, for standard ANNs, the energy consumption is straightforward, as every neuron typically performs MAC operations according to Eq. (21):

$$E_{\text{ANN}} = E_{\text{MAC}} \times \text{FLOP}_{\text{ANN}}, \quad (21)$$

reflecting the uniform use of MAC in all layers.

5.3.6 Results and discussion

Using this methodology, we estimated the per-sample energy costs for a range of ANN and SNN models under identical testing dataset split (see Table [15] for an illustrative subset). In particular:

- **SNN Models:** Achieved lower inference energy under neuromorphic-style MAC+AC assumptions, mainly when firing rates remained low.
- **ANN Models:** Consistently required higher MAC-related energy, reflecting dense floating-point operations.
- **Dataset Differences:** Both Jenin and Amman produce comparable trends, but the absolute energy levels differ due to dataset-specific factors (e.g., intrinsic variations in input content).

These theoretical results align with the rough estimates observed via KerasSpiking (Methodology #1), suggesting that SNNs hold significant theoretical energy-saving potential-particularly when implemented on hardware that exploits spike-based sparsity. However, real-world performance hinges on hardware-specific constraints. Indeed, follow-up tests (Methodology #3) with non-neuromorphic GPUs indicated that SNN advantages may diminish on conventional architectures.

5.3.7 Limitations and future work

Our calculations assume the 45 nm process node and typical energy constants ($E_{\text{MAC}} = 4.6 \text{ pJ}$ and $E_{\text{AC}} = 0.9 \text{ pJ}$), as described in [92] and corroborated by related literature [94]. If implemented on alternative technology nodes or hardware backends, these assumptions may overestimate or underestimate energy consumption. Future work could incorporate device-level measurements from neuromorphic chips (e.g., Intel's Loihi) or integrate design-time tools (e.g., energy-accurate simulators) to refine these estimates further.

5.3.8 Methodology #3: empirical experiments using tesla GPU T4 hardware

We conducted empirical experiments on an **NVIDIA Tesla GPU T4** using the *nvidia-smi* library [84]. This methodology directly monitors real-time power consumption during both *training* and *inference* phases of our ANNs and SNNs, thereby complementing the theoretical estimations presented in Sects. 5.3.1 and 5.3.2.

5.3.9 Experimental setup and empirical results

The experiments were performed on a dedicated server equipped with a Tesla GPU T4. We employed the *nvidia-smi* utility (as documented in the official NVIDIA guide [84]) for power monitoring at one-second intervals. This approach aligns with prior literature on GPU energy measurements, such as in the study [93] and other GPU-based deep learning energy studies [95].

We used *nvidia-smi* in the background to log instantaneous power-draw values (in watts) every second, enabling the accumulation of raw power readings $P(t)$ over time to obtain the total energy consumption in joules (J). Specifically, each power measurement $P(t)$ (in watts) multiplied by 1 s yields $P(t) \times 1 \text{ s}$ joules, and summing across the entire logging interval gives the total GPU energy usage. While our SNNs leverage *cuSPARS* library [84] to manage sparse matrix operations, the GPU effectively processes these workloads as dense layers, which can influence the measured results. Table 16 summarizes the *actual* energy consumption in joules for both training (**Train (J)**) and inference (**Infer (J)**) on the Jenin and Amman datasets. We repeated this procedure for ANN and SNN models.

5.3.10 Discussion and limitations

Results in Table 16 indicate that, despite theoretical analyses (Methodology #1) and #2 predicting lower SNN power usage, our empirical measurements on a Tesla GPU T4 show higher overall energy consumption for SNNs, aligning with previous findings that standard GPUs do not fully exploit spiking sparsity [93, 96–98]. This mismatch compared to the approximate or theoretical estimates from Methodologies #1 and #2 underlines how real-world performance hinges on the underlying hardware architecture. True SNN energy gains would likely emerge on dedicated neuromorphic chips (e.g., Intel Loihi), where event-driven processing is natively supported. In contrast, conventional GPUs incur overheads (e.g., dense-like memory transfers) that mask potential benefits. Although our power logging methodology accurately measures GPU usage during training and inference, several caveats remain: (i) the *nvidia-smi* logging overlooks CPU, DRAM, and other system overheads, focusing primarily on GPU draw; (ii) sampling power at one-second intervals may miss fine-grained fluctuations; and (iii) the Tesla GPU T4 is not a neuromorphic accelerator, so many *spike-based* advantages remain unexploited [93]. Overall, these GPU-centric experiments (Methodology #3) confirm that SNNs do *not* realize the theoretical energy savings observed in spiking-oriented literature when implemented on non-neuromorphic hardware. Nevertheless, measuring real-world behavior provides key insights into hardware-specific constraints and corroborates prior studies on network optimization [93, 95].

5.3.11 Balancing accuracy and power consumption

Our study examines how model accuracy (measured by metrics such as RMSE, nRMSE, and MAE) intersects with energy efficiency, referencing the performance results in Table 11 and the energy data in Tables 13 and 15. While certain SNN configurations can match top-performing ANNs in forecasting precision, others may yield slightly higher errors yet offer notable power savings—an appealing trade-off for applications prioritizing energy efficiency. Moreover, both the KerasSpiking-based estimates (Methodology #1) and the Synops-based theoretical model (Methodology #2) suggest that SNNs excel at reducing energy consumption, particularly on neuromorphic platforms or under conditions of sparse firing activity. However, real-world measurements on the Tesla GPU T4 (Methodology #3) indicate that these theoretical gains may diminish on conventional hardware, where dense numerical operations and overheads like memory transfers negate the benefits of event-driven computation. Consequently, while neuromorphic architectures are poised to unlock SNNs’ full energy-saving potential, standard GPUs may still register higher power usage for spike-based workloads. By highlighting these trade-offs, our study underscores how SNNs can be tailored—balancing accuracy requirements with energy constraints—in diverse domains, including sustainable technologies such as photovoltaic systems.

5.4 Optimal lag selection for solar irradiation forecasting

Accurately forecasting solar irradiation and selecting an appropriate lag size is imperative to ensuring an accurate and power-efficient model. Our empirical investigation focused on finding a balance between forecast accuracy, represented by the nRMSE, and our models’ energy consumption on the neuromorphic Loihi hardware platform.

The study experimented with lag values ranging from 1 to 12 to predict solar irradiation one step ahead (1 h). Our systematic analysis involved calculating total events and Synops to estimate power consumption, relying on established constants for neuromorphic hardware efficiency as detailed in Table 14. The results indicated a clear trade-off between nRMSE and power consumption across lag settings.

This study used the dataset from Jenin, representing hourly solar radiation with an average of 12 sunshine hours per day. Upon a thorough review, as elucidated in Table 17, a lag value of 3 emerged as the most suitable choice. It demonstrated a compelling trade-off between forecast accuracy and power consumption. The rationale behind this decision extends beyond the empirical data; it is supported by the intrinsic characteristics of our dataset and the one step ahead forecasting objective.

By selecting a 3-hour lag, we ensure the model captures sufficient historical information to predict accurately without incorporating redundant data that could lead to over-complexity. This approach optimizes performance and energy efficiency, particularly for solar irradiation forecasting models on neuromorphic hardware platforms. It ensures high accuracy while maintaining low power consumption, making it suitable for sustainable energy systems and allowing the network to operate within the ideal bounds of learning stability and computational demands.

Table 14 Energy constants E_o and E_u (in Nanojoules, nJ) for the hardware platforms used, based on [86, 87]

Hardware Platform	E_o (nJ)	E_u (nJ)
GPU (NVIDIA GTX Titan Black) [90]	0.3	0.3
Loihi [91]	0.02711	0.081

Table 15 Theoretical Energy Consumption (in nJ) per Single Inference Step for SNNs vs. ANNs

Dataset	Network	Layers	Time Series	Neurons	Encoding	Training	Estimated Energy/nJ
Jenin	ANN	LSTM	Multi	ReLU	None	Backprop	109.22
	ANN	LSTM	Uni	ReLU	None	Backprop	79.19
	ANN	MLP	Multi	ReLU	None	Backprop	24.44
	ANN	MLP	Uni	ReLU	None	Backprop	5.30
	ANN	CNN	Multi	ReLU	None	Backprop	67.12
	ANN	CNN	Uni	ReLU	None	Backprop	47.69
	SNN	MLP	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	0.56
	SNN	MLP	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	0.37
	SNN	CNN	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	1.30
	SNN	CNN	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	1.37
	SNN	MLP	Multi	CUBA-LIF	Rate Coding	Bootstrap	6.03
	SNN	MLP	Uni	CUBA-LIF	Rate Coding	Bootstrap	1.35
	SNN	CNN	Multi	CUBA-LIF	Rate Coding	Bootstrap	4.87
	SNN	CNN	Uni	CUBA-LIF	Rate Coding	Bootstrap	3.20
Amman	ANN	LSTM	Multi	ReLU	None	Backprop	109.22
	ANN	LSTM	Uni	ReLU	None	Backprop	79.19
	ANN	MLP	Multi	ReLU	None	Backprop	24.44
	ANN	MLP	Uni	ReLU	None	Backprop	5.30
	ANN	CNN	Multi	ReLU	None	Backprop	67.12
	ANN	CNN	Uni	ReLU	None	Backprop	47.69
	SNN	MLP	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	0.55
	SNN	MLP	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	0.38
	SNN	CNN	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	1.34
	SNN	CNN	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	1.41
	SNN	MLP	Multi	CUBA-LIF	Rate Coding	Bootstrap	6.73
	SNN	MLP	Uni	CUBA-LIF	Rate Coding	Bootstrap	1.74
	SNN	CNN	Multi	CUBA-LIF	Rate Coding	Bootstrap	7.358
	SNN	CNN	Uni	CUBA-LIF	Rate Coding	Bootstrap	3.24

6 Conclusion and future directions

Our comprehensive evaluation of ANNs and SNNs for solar radiation forecasting has demonstrated the potential of SNNs, particularly in terms of energy efficiency. While our comparative analysis, using diverse architectures and encoding strategies as discussed in Sect. 5, showed that SNNs performed competitively in accuracy, they did not consistently surpass the performance of traditional models like LSTM, CNNs, and MLPs across all metrics. However, SNNs exhibited significant advantages in power consumption. Applying advanced methodologies such as the SLAYER algorithm and the Bootstrap conversion approach enabled SNNs to achieve notable results. As evidenced in Table 11, the performance of SNNs, measured by metrics including RMSE, MAE, and nRMSE, was comparable to traditional models while offering superior energy efficiency. This highlights SNNs as a viable and efficient alternative for solar radiation forecasting. This study's findings are significant amid the global push toward integrating PV systems into power grids. This is particularly relevant in Arab countries like Palestine and Jordan, where solar energy is a critical component of sustainable development, as outlined by the United Nations' Sustainable Development Goals [2]. The enhanced computational efficiency and lower power consumption of SNNs make them particularly suitable for environments where PV is the primary energy source. While SNNs offer competitive accuracy, their primary advantage lies in their ability to operate efficiently with limited energy resources. This capability could greatly facilitate the expansion of solar infrastructure in high-insolation areas,

Table 16 Comparison of training and inference energy consumption for ANNs and SNNs on Jenin and Amman datasets in joules (actual empirical measurements on tesla GPU T4)

Dataset	Network	Layers	Time series	Neurons	Encoding	Training	Train (J)	Infer (J)
Jenin	ANN	LSTM	Multi	ReLU	None	Backprop	2572.46	64.68
	ANN	LSTM	Uni	ReLU	None	Backprop	2468.94	65.87
	ANN	MLP	Multi	ReLU	None	Backprop	1785.50	64.38
	ANN	MLP	Uni	ReLU	None	Backprop	1458.13	55.55
	ANN	CNN	Multi	ReLU	None	Backprop	2089.94	64.48
	ANN	CNN	Uni	ReLU	None	Backprop	2146.45	96.92
	SNN	MLP	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	6112.74	84.08
	SNN	MLP	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	4702.47	84.14
	SNN	CNN	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	9705.72	82.10
	SNN	CNN	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	6690.08	81.89
	SNN	MLP	Multi	CUBA-LIF	Rate coding	Bootstrap	3949.64	24.53
	SNN	MLP	Uni	CUBA-LIF	Rate coding	Bootstrap	4049.67	23.56
	SNN	CNN	Multi	CUBA-LIF	Rate coding	Bootstrap	6527.20	25.90
	SNN	CNN	Uni	CUBA-LIF	Rate coding	Bootstrap	6379.53	25.56
Amman	ANN	LSTM	Multi	ReLU	None	Backprop	2601.61	65.47
	ANN	LSTM	Uni	ReLU	None	Backprop	2502.28	64.68
	ANN	MLP	Multi	ReLU	None	Backprop	1712.34	64.38
	ANN	MLP	Uni	ReLU	None	Backprop	1652.19	64.78
	ANN	CNN	Multi	ReLU	None	Backprop	2122.66	64.58
	ANN	CNN	Uni	ReLU	None	Backprop	2109.70	64.28
	SNN	MLP	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	6179.97	87.20
	SNN	MLP	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	4849.50	87.40
	SNN	CNN	Multi	Sigma-Delta LIF	Sigma-Delta	SLAYER	9683.88	87.30
	SNN	CNN	Uni	Sigma-Delta LIF	Sigma-Delta	SLAYER	7014.45	90.62
	SNN	MLP	Multi	CUBA-LIF	Rate coding	Bootstrap	3934.12	23.05
	SNN	MLP	Uni	CUBA-LIF	Rate coding	Bootstrap	4009.93	26.46
	SNN	CNN	Multi	CUBA-LIF	Rate coding	Bootstrap	6703.01	26.09
	SNN	CNN	Uni	CUBA-LIF	Rate coding	Bootstrap	6579.49	25.99

Table 17 Optimal lag selection for power-efficient and performant SNNs on Loihi platform

Model	lag=1	lag=2	lag=3	lag=4	lag=5	lag=6	lag=7	lag=8	lag=9	lag=10	lag=11	lag=12
ANN -LSTM-Multivariate (nRMSE)	0.064	0.048	0.047	0.047	0.047	0.047	0.047	0.047	0.047	0.047	0.047	0.047
ANN -LSTM-Multivariate (Power Consumption on Loihi [nJ])	201.34	222.16	243.00	263.80	940.46	3,967	4,216	4,465	4,714	4,963	5,212	5,460
SNN CNN Multivariate (Sigma-Delta LIF) (nRMSE)	0.063	0.049	0.048	0.048	0.049	0.048	0.048	0.049	0.049	0.048	0.076	0.048
SNN CNN Multivariate (Sigma-Delta LIF) (Power Consumption on Loihi [nJ])	6.24	9.36	12.29	15.50	18.83	21.43	24.54	27.95	30.22	34.61	36.47	40.25
SNN CNN Univariate (CUBA-LIF) (nRMSE)	0.910	0.067	0.057	0.057	0.057	0.058	0.057	0.057	0.058	0.057	0.057	0.057
SNN CNN Univariate (CUBA-LIF) (Power Consumption on Loihi [nJ])	5.25	8.91	10.29	10.58	11.59	12.06	13.33	14.38	10.74	14.62	11.04	14.48

supporting global low-carbon goals and regional energy independence. The experimental data underscored the potential of SNNs to operate with reduced computational overhead and improved compatibility with neuromorphic hardware, as evidenced in Table 12, suggesting a pivotal role for these networks in resource-constrained applications. Our analyses have revealed crucial insights into the trade-offs between model sparsity, power efficiency, and predictive accuracy. This presents a balanced approach to network design that could inform future developments in the field.

Several avenues for future research are proposed to build upon this study's findings. First, designing and developing hybrid models that combine the strengths of ANNs and SNNs could be explored. Such hybrid architectures may leverage the high accuracy of ANNs and the energy efficiency of SNNs, potentially achieving superior performance in both metrics. Second, improving the learning algorithms for SNNs remains an open challenge. Investigating advanced training techniques and optimization methods may enhance the convergence and generalization capabilities of SNNs, making them more robust for complex forecasting tasks. Third, applying the methodologies developed in this study to other domains beyond solar radiation forecasting could demonstrate the versatility and robustness of SNNs. This approach could benefit fields such as wind energy prediction, load forecasting in smart grids, or even non-energy-related time series forecasting tasks. Additionally, further experimentation on neuromorphic hardware platforms, such as Intel's Loihi, can provide practical insights into the deployment and scalability of SNNs in real-world applications. Exploring hardware-software co-design strategies may also optimize the performance of SNNs on such platforms. Finally, while our study used ARIMA models as a benchmark for evaluating the predictive performance of ANNs and SNNs, future research could incorporate more advanced statistical models or ensemble methods to provide a more comprehensive comparison and further validate the advantages of neural network approaches.

In conclusion, integrating SNNs into the solar PV forecasting domain represents a transformative advancement for energy systems, particularly in politically and economically sensitive regions. By leveraging SNNs' unique capabilities and continuing to innovate in neural network technologies, we can better address the complex challenges of sustainable energy development.

Author Contributions Bahgat Ayasi was involved in resources, software, methodology, data curation, investigation, and writing—original draft, Iago X. Vázquez contributed to resources and investigation. Angel M. Garcia-Vico took part in Writing – review & editing, Writing – original draft, Validation, Supervision, Software, Resources, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. Cristobal J. Carmona performed Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization. Mohammed Saleh was responsible for resources and formal analysis.

Funding This paper was supported by the Ministry of Science and Innovation with project PID2023-149511OB-I00

Data Availability The experimental dataset on solar radiation and meteorological parameters used in this study is publicly available in the National Solar Radiation Database (NSRDB). It can be accessed through the following link: <https://nsrdb.nrel.gov/data-viewer>.

Declarations

Conflict of interest All authors describe no potential conflict of interest.

References

1. Hamada S, Ghodieh A (2021) Mapping of solar energy potential in the west bank, palestine using geographic information systems. *Papers Appl Geogr* 7(3):256–273. <https://doi.org/10.1080/23754931.2020.1870540>
2. United Nations: The 17 Goals. <https://sdgs.un.org/goals>. Accessed: 15-Jan-2024 (2015)

3. Omar MA, Mahmoud MM (2018) Grid connected pv home systems in palestine: a review on technical performance, effects and economic feasibility. *Renew Sustain Energy Rev* 82:2490–2497. <https://doi.org/10.1016/j.rser.2017.09.008>
4. Nassar YF, Alsadi SY (2019) Assessment of solar energy potential in gaza strip-palestine. *Sustain Energy Technol Assess* 31:318–328. <https://doi.org/10.1016/j.seta.2018.12.010>
5. Fan Z, Yan Z, Wen S (2023) Deep learning and artificial intelligence in sustainability: A review of sdgs, renewable energy, and environmental health. *Sustainability* 15(18):13493. <https://doi.org/10.3390/su151813493>
6. Khatib T, Bazyan A, Assi H, Malhis S (2021) Palestine energy policy for photovoltaic generation: current status and what should be next? *Sustainability (Switzerland)* 13(5):1–19. <https://doi.org/10.3390/su13052996>
7. Biancardi A, D'Adamo I, D'Amore F, Moretti R (2024) Prosumers as drivers of sdg7 in palestine: net-benefit analysis of grid-connected photovoltaic systems. *Utilities Policy* 87:101730. <https://doi.org/10.1016/j.jup.2024.101730>
8. Natsheh EM (2016) Power generation of solar pv systems in palestine. *Appl Solar Energy* 52:193–196. <https://doi.org/10.3103/S0003701X16030105>
9. Said N, Alsamamra H (2019) An overview of green buildings potential in palestine. *Int J Sustain Green Energy* 8(2): 11. <https://doi.org/10.11648/j.ijrse.20190802.11>
10. Rohde F, Gossen M, Wagner J, Santarius T (2021) Sustainability challenges of artificial intelligence and policy implications. *Ökologisches Wirtschaften-Fachzeitschrift* 36(O1):36–40. <https://doi.org/10.1007/s00506-021-00748-3>
11. Lohn AJ, Musser M (December 2021) Ai and compute: How much longer can computing power drive artificial intelligence progress? Technical report, Center for Security and Emerging Technology. <https://doi.org/10.51593/2021CA009>. <https://cset.georgetown.edu/wp-content/uploads/CSET-2021-CA009-AI-and-Compute.pdf>
12. Pachot A, Patissier C (2023) Toward sustainable artificial intelligence: An overview of environmental protection uses and issues. *Green and Low-Carbon Economy* 00(00), 1–8 <https://doi.org/10.47852/bonviewGLCE3202608>
13. Energy Sector Management Assistance Program: Global Photovoltaic Power Potential by Country. World Bank. Available online: <https://doi.org/10.1596/34102> (2020). <https://doi.org/10.1596/34102>
14. Imamkulieva E (2022) In: Name, E. (ed.) *Clean Energy in the Arab Countries*, pp. 58–72. Monograph Compiler Publications, Publisher Location
15. Ibrahim IA (2023) Energy transition and sustainable development goal 7: a legal analysis in the context of the arab world. *J World Energy Law Bus* 16(2):77–90. <https://doi.org/10.1080/23754931.2020.1870540>
16. Chodakowska E, Nazarko J, Nazarko Rabayah HS, Abende RM, Alawneh R (2023) Arima models in solar radiation forecasting in different geographic locations. *Energies* 16(13):5029. <https://doi.org/10.3390/en16135029>
17. Braik A, Makhalfih A, Hamid ASA, Sopian K, Ibrahim A (2022) Impact of photovoltaic grid-tied systems on national grid power factor in palestine. *AIMS Energy* 10(2):236–253. <https://doi.org/10.3934/energy.2022013>
18. Yang D, Jirutitijaroen P, Walsh WM (2012) Hourly solar irradiance time series forecasting using cloud cover index. *Sol Energy* 86(12):3531–3543. <https://doi.org/10.1016/j.solener.2012.07.029>
19. Alsharif MH, Younes MK, Kim J (2019) Time series arima model for prediction of daily and monthly average global solar radiation: The case study of seoul, south korea. *Symmetry* 11(2):240. <https://doi.org/10.3390/sym11020240>
20. Zamani Sabzi H, King JP, Abudu S (2017) Developing an intelligent expert system for streamflow prediction, integrated in a dynamic decision support system for managing multiple reservoirs: A case study. *Expert Syst Appl* 83:145–163. <https://doi.org/10.1016/j.eswa.2017.04.039>
21. Freeman BS, Taylor G, Gharabaghi B, Thé J (2018) Forecasting air quality time series using deep learning. *J Air Waste Manag Assoc* 68(8):866–886. <https://doi.org/10.1080/10962247.2018.1459956>
22. Zafirakis D, Tzanes G, Kaldellis JK (2019) Forecasting of wind power generation with the use of artificial neural networks and support vector regression models. *Energy Proc* 159:509–514. <https://doi.org/10.1016/j.egypro.2018.12.007>
23. Lara-Benítez P, Carranza-García M, Riquelme JC (2021) An experimental review on deep learning architectures for time series forecasting. *Int J Neural Syst* 31(03):2130001. <https://doi.org/10.1142/s0129065721300011>
24. Strubell E, Ganesh A, McCallum A (2020) Energy and policy considerations for modern deep learning research. *Proc AAAI Conf Artif Intell* 34:13693–13696
25. Lacoste A, Luccioni A, Schmidt V, Dandres T (2019) Quantifying the carbon emissions of machine learning. *arXiv preprint arXiv:1910.09700*
26. Schliebs S, Kasabov N (2013) Evolving spiking neural network-a survey. *Evol Syst* 4(2):87–98. <https://doi.org/10.1007/s12530-013-9074-9>
27. Maass W (1997) Networks of spiking neurons: the third generation of neural network models. *Neural Netw* 10(9):1659–1671
28. Ghosh-Dastidar S, Adeli H (2009) Spiking neural networks. *Int J Neural Syst* 19(04):295–308. <https://doi.org/10.1142/s0129065709002002>
29. Paugam-Moisy H, Bohtë SM (2012) Computing with Spiking Neuron Networks, pp. 335–376. https://doi.org/10.1007/978-3-540-92910-9_10
30. Kulkarni S, Simon SP, Sundareswaran K (2013) A spiking neural network (snn) forecast engine for short-term electrical load forecasting. *Appl Soft Comput* 13(8):3628–3635. <https://doi.org/10.1016/j.asoc.2013.04.007>

31. Reid D, Hussain AJ, Tawfik H (2014) Financial time series prediction using spiking neural networks. *PLoS ONE* 9(8):103656. <https://doi.org/10.1371/journal.pone.0103656>
32. Sun G, Chen T, Wei Z, Sun Y, Zang H, Chen S (2016) A carbon price forecasting model based on variational mode decomposition and spiking neural networks. *Energies* 9(1):54. <https://doi.org/10.3390/en9010054>
33. Zhang A, Zhu W, Li J (2019) Spiking echo state convolutional neural network for robust time series classification. *IEEE Access* 7:4927–4935. <https://doi.org/10.1109/ACCESS.2018.2887354>
34. Fang H, Shrestha A, Qiu Q (2020) Multivariate time series classification using spiking neural networks. In: 2020 international joint conference on neural networks (IJCNN), pp. 1–6. <https://doi.org/10.1109/IJCNN48605.2020.9206751>. IEEE
35. Matenczuk K, Kozina A, Markowska A, Czerniachowska K, Kaczmarczyk K, Golec P, Hernes M, Lutosławski K, Kozierkiewicz A, Pietranik M, Rot A, Dyvak M (2021) Financial time series forecasting: Comparison of traditional and spiking neural networks. *Proc Comput Sci* 192:5023–5029. <https://doi.org/10.1016/j.procs.2021.09.280>
36. González Sopeña JM, Pakrashi V, Ghosh B (2022) A spiking neural network based wind power forecasting model for neuromorphic devices. *Energies* 15(19):97256. <https://doi.org/10.3390/en15197256>
37. Intel's Neuromorphic Computing Lab: Lava: A Software Framework for Neuromorphic Computing. <http://lava-nc.org> and <https://github.com/lava-nc/lava>. Accessed January 1, 2024 (2021)
38. Lai CS, Zhong C, Pan K, Ng WWY, Lai LL (2021) A deep learning based hybrid method for hourly solar radiation forecasting. *Expert Syst Appl* 177:114941. <https://doi.org/10.1016/j.eswa.2021.114941>
39. Box GE, Jenkins GM, Reinsel GC, Ljung GM (2015) Time series analysis: forecasting and control. Wiley, Hoboken, NJ
40. Ervural BC, Beyca OF, Zaim S (2016) Model estimation of arma using genetic algorithms: a case study of forecasting natural gas consumption. *Proc Soc Behav Sci* 235:537–545. <https://doi.org/10.1016/j.sbspro.2016.11.010>
41. Cayir Ervural B, Ervural B (2018) Improvement of grey prediction models and their usage for energy demand forecasting. *J Intell Fuzzy Syst* 34(4):2679–2688
42. Hochreiter S, Schmidhuber J (1997) Long short-term memory. *Neural Comput* 9(8):1735–1780
43. Jebb AT, Tay L (2016) Introduction to time series analysis for organizational research: methods for longitudinal analyses. *Organ Res Methods* 20(1):61–94. <https://doi.org/10.1177/1094428116668035>
44. Siami-Namini S, Tavakoli N, Namin AS (2019) A comparative analysis of forecasting financial time series using ARIMA, LSTM, and BiLSTM. <http://export.arxiv.org/pdf/1911.09512>
45. Yan R, Liao J, Yang J, Sun W, Nong M, Li F (2021) Multi-hour and multi-site air quality index forecasting in beijing using cnn, lstm, cnn-lstm, and spatiotemporal clustering. *Expert Syst Appl* 169:114513. <https://doi.org/10.1016/j.eswa.2020.114513>
46. Radzi PNLM, Akhter MN, Mekhilef S, Shah NM (2023) Review on the application of photovoltaic forecasting using machine learning for very short-to long-term forecasting. *Sustainability* 15(4):2942. <https://doi.org/10.3390/su15042942>
47. Sharifzadeh M, Sikinioti-Lock A, Shah N (2019) Machine-learning methods for integrated renewable power generation: a comparative study of artificial neural networks, support vector regression, and gaussian process regression. *Renew Sustain Energy Rev* 108:513–538. <https://doi.org/10.1016/j.rser.2019.03.040>
48. Rana M, Koprinska I, Agelidis VG (2016) Univariate and multivariate methods for very short-term solar photovoltaic power forecasting. *Energy Convers Manag* 121:380–390. <https://doi.org/10.1016/j.enconman.2016.05.025>
49. Ahmed R, Sreeram V, Mishra Y, Arif MD (2020) A review and evaluation of the state-of-the-art in pv solar power forecasting: techniques and optimization. *Renew Sustain Energy Rev* 124:109792. <https://doi.org/10.1016/j.rser.2020.109792>
50. Lim B, Zohren S (2021) Time-series forecasting with deep learning: a survey. *Phil Trans R Soc A* 379(2194):20200209. <https://doi.org/10.1098/rsta.2020.0209>
51. Wang K, Qi X, Liu H (2019) A comparison of day-ahead photovoltaic power forecasting models based on deep learning neural network. *Appl Energy* 251:113315. <https://doi.org/10.1016/j.apenergy.2019.113315>
52. Ren X, Zhang F, Zhu H, Liu Y (2022) Quad-kernel deep convolutional neural network for intra-hour photovoltaic power forecasting. *Appl Energy* 319:119682. <https://doi.org/10.1016/j.apenergy.2022.119682>
53. Qing X, Niu Y (2018) Hourly day-ahead solar irradiance prediction using weather forecasts by lstm. *Energy* 148:461–468
54. Yang D (2019) A guideline to solar forecasting research practice: reproducible, operational, probabilistic or physically-based, ensemble, and skill (ropes). *J Renew Sustain Energy* 11(2):102
55. Yan K, Shen H, Wang L, Zhou H, Xu M, Mo Y (2020) Short-term solar irradiance forecasting based on a hybrid deep learning methodology. *Information* 11(1):32
56. Obiora CN, Hasan AN, Ali A, Alajarmeh N (2021) Forecasting hourly solar radiation using artificial intelligence techniques. *IEEE Canad J Elect Comput Eng* 44(4):497–508
57. Mukhtar M, Oluwasanmi A, Yimen N, Qinxu Z, Ukwuoma CC, Ezurike B, Bamisile O (2022) Development and comparison of two novel hybrid neural network models for hourly solar radiation prediction. *Appl Sci* 12(3):1435
58. Gerstner W, Kistler WM, Naud R, Paninski L (2014) Neuronal dynamics: from single neurons to networks and models of cognition. Cambridge University Press, Cambridge, UK

59. Gerstner W, Kistler WM (2002) Spiking Neuron Models: Single Neurons, Populations, Plasticity. Cambridge University Press, Cambridge, UK. <http://catdir.loc.gov/catdir/samples/cam031/2002067657.pdf>
60. Paugam-Moisy H (2006) Spiking neuron networks a survey. Technical report, EPFL-REPORT-83371. <http://publications.idiap.ch/downloads/reports/2006/paugam-idiap-rr-06-11.pdf>
61. Izhikevich EM (2003) Simple model of spiking neurons. *IEEE Trans Neural Netw* 14(6):1569–1572. <https://doi.org/10.1109/TNN.2003.820440>
62. Zambrano D, Bohte SM (2016) Fast and efficient asynchronous neural computation with adapting spiking neural networks. arXiv preprint [arXiv:1609.02053](https://arxiv.org/abs/1609.02053)
63. Brette R, Rudolph M, Carnevale T, Hines M, Beeman D, Bower JM, Destexhe A (2007) Simulation of networks of spiking neurons: a review of tools and strategies. *J Comput Neurosci* 23:349–398
64. Shrestha SB, Orchard G (2018) SLAYER: Spike layer error reassignment in time. <https://arxiv.org/pdf/1810.08646>
65. Davies M (2021) Taking neuromorphic computing to the next level with Loihi2. Intel Labs' Loihi
66. Lee C, Sarwar SS, Panda P, Srinivasan G, Roy K (2020) Enabling spike-based backpropagation for training deep neural network architectures. *Front Neurosci* 14:119. <https://doi.org/10.3389/fnins.2020.00119>
67. Diehl PU, Cook M (2015) Unsupervised learning of digit recognition using spike-timing-dependent plasticity. *Front Comput Neurosci* 9:99. <https://doi.org/10.3389/fncom.2015.00099>
68. Long L, Liu Q, Peng H, Yang Q, Luo X, Wang J, Song X (2022) A time series forecasting approach based on nonlinear spiking neural systems. *Int J Neural Syst* 32(08):204. <https://doi.org/10.1142/s0129065722500204>
69. Kulkarni S, Rajendran B (2018) Spiking neural networks for handwritten digit recognition-supervised learning and network optimization. *Neural Netw* 103:118–127. <https://doi.org/10.1016/j.neunet.2018.03.019>
70. Deng L, Wu Y, Hu X, Liang L, Ding Y, Li G, Zhao G, Li P, Xie Y (2020) Rethinking the performance comparison between snns and anns. *Neural Netw* 121:294–307. <https://doi.org/10.1016/j.neunet.2019.09.005>
71. Tan C, Šarlija M, Kasabov N (2021) Neurosense: short-term emotion recognition and understanding based on spiking neural network modelling of spatio-temporal eeg patterns. *Neurocomputing* 434:137–148. <https://doi.org/10.1016/j.neucom.2020.12.098>
72. Wang H, Xue W, Liu Y, Peng J, Jiang H (2020) Probabilistic wind power forecasting based on spiking neural network. *Energy* 196:117072. <https://doi.org/10.1016/j.energy.2020.117072>
73. Zhan G, Song Z, Fang T, Zhang Y, Le S, Zhang X, Kang X (2021) Applications of spiking neural network in brain computer interface. In: 2021 9th International Winter Conference on Brain-Computer Interface (BCI), pp. 1–6. <https://doi.org/10.1109/BCI51272.2021.9385361>. IEEE
74. Capizzi G, Sciuto GL, Napoli C, Woźniak M, Susi G (2020) A spiking neural network-based long-term prediction system for biogas production. *Neural Netw* 129:271–279. <https://doi.org/10.1016/j.neunet.2020.06.001>
75. Thangaraj VK, Nachimuthu DS, Francis VAR (2023) Wind speed forecasting at wind farm locations with an unique hybrid pso-alo based modified spiking neural network. *Energy Syst* 4:1–29. <https://doi.org/10.1007/s12667-023-00518-4>
76. Manna DL, Vicente-Sola A, Kirkland P, Bihl TJ, Di Caterina G (2023) An approach to time series forecasting with spiking neural networks. Preprint
77. Gao Y, Wang J, Guo L, Peng H (2024) Short-term photovoltaic power prediction using nonlinear spiking neural p systems. *Sustainability* 16(4):1709. <https://doi.org/10.3390/su16041709>
78. Sengupta M, Xie Y, Lopez A, Habte A, Maclaurin G, Shelby J (2018) The national solar radiation data base (nsrdb). *Renew Sustain Energy Rev* 89:51–60
79. Guo W, Fouda ME, Eltawil AM, Salama KN (2021) Neural coding in spiking neural networks: a comparative study for robust neuromorphic systems. *Front Neurosci* 15:638474. <https://doi.org/10.3389/fnins.2021.638474>
80. Nair MV, Indiveri G (2019) An ultra-low power sigma-delta neuron circuit. In: 2019 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–5. IEEE
81. Kim Y, Park H, Moitra A, Bhattacharjee A, Venkatesha Y, Panda P (2022) Rate coding or direct coding: Which one is better for accurate, robust, and energy-efficient spiking neural networks? In: 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 71–75. IEEE
82. Guo W, Fouda ME, Eltawil AM, Salama KN (2021) Neural coding in spiking neural networks: a comparative study for robust neuromorphic systems. *Front Neurosci* 15:638474
83. Pan Z, Wu J, Zhang M, Li H, Chua Y (2019) Neural population coding for effective temporal classification. In: 2019 International Joint Conference on Neural Networks (IJCNN), pp. 1–8. IEEE
84. Corporation N (2023) NVIDIA System Management Interface. <http://developer.download.nvidia.com/compute/DCGM/docs/nvidia-smi-367.38.pdf>. Accessed: 20 February
85. Liu L, Jiang H, He P, Chen W, Liu X, Gao J, Han J (2019) On the variance of the adaptive learning rate and beyond. arXiv preprint [arXiv:1908.03265](https://arxiv.org/abs/1908.03265)
86. Kucik AS, Meoni G (2021) Investigating spiking neural networks for energy-efficient on-board ai applications: A case study in land cover and land use classification. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 2020–2030
87. Research AB (2024) KerasSpiking - Estimating model energy. <https://www.nengo.ai/keras-spiking/examples/model-energy.html>. Last accessed on 12/04/2024

88. Ali HAH, Dabbous A, Ibrahim A, Valle M (2023) Assessment of recurrent spiking neural networks on neuromorphic accelerators for naturalistic texture classification. In: 2023 18th Conference on Ph. D Research in Microelectronics and Electronics (PRIME)
89. Herbozo Contreras LF, Huang Z, Yu L, Nikpour A, Kavehei O (2024) Biological plausible algorithm for seizure detection: Toward ai-enabled electroceuticals at the edge. *APL Machine Learning* 2(2)
90. Degnan B, Marr B, Hasler J (2015) Assessing trends in performance per watt for signal processing applications. *IEEE Trans Very Large Scale Integr VLSI Syst* 24(1):58–66
91. Davies M, Srinivasa N, Lin TH, Chinya G, Cao Y, Choday SH, Dimou G, Joshi P, Imam N, Jain S et al (2018) Loihi: a neuromorphic manycore processor with on-chip learning. *IEEE Micro* 38(1):82–99. <https://doi.org/10.1109/MM.2018.112130359>
92. Zhu RJ, Wang Z, Gilpin L, Eshraghian JK (2024) Autonomous driving with spiking neural networks. *ArXiv arXiv:2405.19687*
93. Vidya Wicaksana Putra R, Shafique M (2020) FSpINN: An Optimization Framework for Memory-and Energy-Efficient Spiking Neural Networks. *arXiv e-prints*, [arXiv:2007.00000](https://arxiv.org/abs/2007.00000)
94. Yao M, Zhao G, Zhang H, Hu Y, Deng L, Tian Y, Li G (2023) Attention spiking neural networks. *IEEE Trans Pattern Anal Mach Intell* 45(8):9393–9410
95. Han S, Mao H, Dally WJ (2015) Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. *arXiv preprint arXiv:1510.00149*
96. Pfeiffer M, Pfeil T (2018) Deep learning with spiking neurons: opportunities and challenges. *Front Neurosci* 12:409662. <https://doi.org/10.3389/fnins.2018.409662>
97. Roy K, Jaiswal A, Panda P (2019) Towards spike-based machine intelligence with neuromorphic computing. *Nature* 575(7784):607–617. <https://doi.org/10.1038/s41586-019-1677-2>
98. Davies M, Wild A, Orchard G, Sandamirskaya Y, Guerra GAF, Joshi P, Rusbud SR (2021) Advancing neuromorphic computing with loihi: a survey of results and outlook. *Proc IEEE* 109(5):911–934. <https://doi.org/10.1109/JPROC.2021.3058954>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Bahgat Ayasi^{1,7}  · **Iago X. Vázquez**²  · **Mohammed Saleh**³  · **Angel M. Garcia-Vico**^{4,5}  · **Cristóbal J. Carmona**^{4,5,6} 

✉ Bahgat Ayasi
ba000034@red.ujaen.es

Iago X. Vázquez
ixvazquez@itcl.es

Mohammed Saleh
msaleh@pass.ps

Angel M. Garcia-Vico
agvico@ujaen.es

Cristóbal J. Carmona
ccarmona@ujaen.es

- ¹ Computer Science Department, University of Jaén, Campus Las Lagunillas s/n, 23071 Jaén, Andalucía, Spain
- ² ITCL Technology Centre, 09001 Burgos, Spain
- ³ Information Technology Center, Al-Istiqlal University, Jericho, Palestine
- ⁴ Andalusian Research Institute in Data Science and Computational Intelligence (DaSCI), University of Jaén, E-23071 Jaén, Spain
- ⁵ Department of Computer Science, University of Jaén, E-23071 Jaén, Spain
- ⁶ Leicester School of Pharmacy, De Montfort University, LE1 7RH Leicester, United Kingdom
- ⁷ Computer Science Department, Arab American University(AAUP), 240, Jenin, Palestine